CRTM — Cyber-Resilience Tuple Model

Program code listing, software version 1.0.0

Digital supplement (Code S1) to the article “A Set-Theoretic Tuple Model with Level-Consistent Normalization and Non-Compensatory Aggregation for Assessing the Cyber Resilience of Critical Infrastructure”, Information (MDPI), 2026. License: MIT (code), CC BY 4.0 (data, figures).
Generated 2026-10-27 by tools/build_listing.py.

File	Lines	SHA-256
crtm/__init__.py	17	3ae540ee0f29d1debd1b4287a905224af540c529b37bf9718aead8571cf95bf1
crtm/aggregation.py	129	4e24393c78ddc962566f65562c9cc72a0ea9458455db036dcb99a6ace4211895
crtm/evaluation.py	104	0a79e58026043afadc2e276edcfc04efdbf7867c7eac01df9a2f768b48f6654f
crtm/figures.py	354	ea1428af53194a91f5f6a77edb2642a4b78fffd05f7b1b7c460a083672b5dbb8
crtm/hierarchy.py	156	5b7a63c7ec41384a3823f356bf7d8b6e37707bceba21a102e28f348e0478cfae
crtm/metrics.py	112	5961d698e7d33c8a2a78a1ded18ff857508816f22963e76a96175864c2e6bfd5
crtm/normalization.py	172	f186802d42fac919b82b897f5f0a4726d36428c321744928b954579872bb8300
crtm/scale.py	51	508915894eeb1f71f1feac4bdee965d73915ee5ca0ef0b0a646c9363515a9f83
crtm/sensitivity.py	37	70d56785106a40f79d211a544dac49e05548a58a56ca65a61a1dd7604bb3c060
crtm/uncertainty.py	40	c5945d915d6c9443c7b3e178f46f317b97e8cd63439ee1524c78e9966d982750
scripts/reproduce.py	349	57ecadde9ee75820d2df3069524dd14f728cf2fa66a689acf6974d0512c360bd
tests/conftest.py	4	c50c8305f585c9551191788e47ab0cbebac1b54229902d72a7bd11ce8f5ae4b5
tests/test_aggregation.py	88	51a332a0e142ab92c235a4da1b58f3ffa12fa945dd8a2e72793c1243110b1f7b
tests/test_hierarchy.py	72	ff5b820a52f5a9a357136805a9b96b114986b57de313820f73054ee5f96ac3e9
tests/test_normalization.py	90	95a42f184f72f638641a5cae8e01845af250c54e00fdab4de51327844013cd54
tests/test_reproduction.py	13	d5277bea807079b4d3d93c7d02222b562ef421b7809af69b6a869a89b31945de
tools/build_listing.py	151	5b776746f47660ccff92a653dbd03080e2d774f5efee630344a6fb5407056523
tools/build_notebook.py	342	3612ba1cd3b4c3f9a8df1becee447ca9c3d9920072337cd343de983d325afbd4

18 files, 2281 lines

crtm/__init__.py

SHA-256 3ae540ee0f29d1debd1b4287a905224af540c529b37bf9718aead8571cf95bf1

```
"""crtm — Cyber-Resilience Tuple Model.

Reference implementation accompanying the article
"A Set-Theoretic Tuple Model with Level-Consistent Normalization and
Non-Compensatory Aggregation for Assessing the Cyber Resilience of
Critical Infrastructure" (Information, MDPI).
"""
from .scale import Y, LEVEL_NAMES, LEVEL_ABBR, level, level_abbr, level_name, band
from .normalization import ClassTable, ClassTableError, grid_check, is_consistent
from .aggregation import additive, gate, hard_gate, power, capped, missing_enclosure
from .hierarchy import (TASKS, GOALS, TECHNIQUES, Projection, iota, iota_inv, check_code,
                        jaccard_distance, jaccard_index, poset_size)
from .metrics import Metric, Action, load_metrics, load_scenarios, DATA_DIR
from .evaluation import evaluate_action, evaluate_scenario, score_vector, dominates
from .uncertainty import dirichlet_moments, sample_scores, SEED, N_SAMPLES

__version__ = "1.0.0"
```

crtm/aggregation.py

SHA-256 4e24393c78ddc962566f65562c9cc72a0ea9458455db036dcb99a6ace4211895

```
"""Non-compensatory aggregation operators (Section 6).

All operators take a vector m in [0, 1]^q of normalized metrics, a weight
vector w on the simplex and a set K of core (knockout) positions
(0-based indices).

  additive     S_w(m)                                  (weighted mean)
  hard_gate    mu^Phi   = Phi_K(m) S_w(m)              (eq. 14)
  power        mu^(g)   = (prod_K m_k)^g S_w(m)        (eq. 15)
  capped       mu^[th]  = min{S_w, min_K m_k / th_k}   (eq. 16)
  missing      mu_O and the enclosure of Proposition 6 (eqs. 17-18)
"""
from __future__ import annotations

from typing import Iterable, Sequence

import numpy as np


def _w(w, q: int | None = None) -> np.ndarray:
    w = np.asarray(w, dtype=float)
    if np.any(w < 0) or not np.isclose(w.sum(), 1.0, atol=1e-9):
        raise ValueError("weights must be nonnegative and sum to 1")
    if q is not None and w.size != q:
        raise ValueError("weights and metrics differ in length")
    return w


def _theta(theta, K: Sequence[int]) -> np.ndarray:
    th = np.broadcast_to(np.asarray(theta, dtype=float), (len(K),))
    if np.any(th <= 0) or np.any(th > 1):
        raise ValueError("thresholds theta_k must lie in (0, 1]")
    return th


def additive(m, w) -> float:
    """Weighted arithmetic mean S_w(m)."""
    m = np.asarray(m, dtype=float)
    return float(_w(w, m.size) @ m)


def gate(m, K: Iterable[int]) -> int:
    """Stop factor Phi_K(m) = prod_{k in K} 1[m_k > 0]."""
    m = np.asarray(m, dtype=float)
    return int(all(m[k] > 0 for k in K))


def hard_gate(m, w, K: Iterable[int] = ()) -> float:
    """Hard-gated score mu^Phi (eq. 14)."""
    return gate(m, K) * additive(m, w)


def power(m, w, K: Sequence[int], gamma: float = 1.0) -> float:
    """Power-discounted score mu^(gamma) (eq. 15)."""
    m = np.asarray(m, dtype=float)
    core = np.prod([m[k] for k in K]) if len(K) else 1.0
    return float(core ** gamma * additive(m, w))


def capped(m, w, K: Sequence[int], theta=0.5) -> float:
    """Capped knockout operator mu^[theta] (Definition 9, eq. 16)."""
    m = np.asarray(m, dtype=float)
    s = additive(m, w)
    if not len(K):
        return s
    th = _theta(theta, K)
    return float(min(s, min(m[k] / t for k, t in zip(K, th))))


def compensation_capacity_additive(w, k: int) -> float:
    """C_k(S_w) = 1 - w_k (Section 6.1)."""
    return float(1.0 - np.asarray(w, dtype=float)[k])


# ------------------------------------------------------------- missing data
def missing_enclosure(m, w, K: Sequence[int] = (), theta=None) -> dict:
    """Renormalized score and enclosure for missing metrics (Proposition 6).

    Missing metrics are NaN in ``m``. Returns a dict with completeness kappa,
    renormalized score mu_O (eq. 17), lower/upper bound of (18), and the
    status. A missing core metric gives status 'not computed'.
    """
    m = np.asarray(m, dtype=float)
    w = _w(w, m.size)
    obs = ~np.isnan(m)
```

```
    if any(not obs[k] for k in K):
        return {"status": "not computed: missing core metric", "kappa": float(w[obs].sum()),
                "mu_O": np.nan, "lower": np.nan, "upper": np.nan}
    kappa = float(w[obs].sum())
    if kappa <= 0:
        raise ValueError("no observed metric carries weight")
    g = gate(np.where(obs, m, 1.0), K)
    mu_o = g * float(w[obs] @ m[obs]) / kappa
    lower = kappa * mu_o
    upper = kappa * mu_o + (1 - kappa) if g else 0.0
    if theta is not None and len(K):
        th = _theta(theta, K)
        cap = min(m[k] / t for k, t in zip(K, th))
        lower, upper, mu_o_cap = min(lower, cap), min(upper, cap), min(mu_o, cap)
    else:
        mu_o_cap = mu_o
    return {"status": "ok" if kappa == 1 else "enclosure", "kappa": kappa,
            "mu_O": float(mu_o_cap), "lower": float(lower), "upper": float(upper),
            "max_deviation": float((1 - kappa) * max(mu_o, 1 - mu_o)) if g else 0.0}


# ------------------------------------------------------------ property tests
def is_idempotent(op, q: int, ts=np.linspace(0, 1, 101)) -> bool:
    """Numerically test A(t, ..., t) = t."""
    return bool(all(np.isclose(op(np.full(q, t)), t) for t in ts))


def has_veto(op, q: int, k: int, trials: int = 200, seed: int = 0) -> bool:
    """Numerically test m_k = 0 => A(m) = 0 on random points."""
    rng = np.random.default_rng(seed)
    for _ in range(trials):
        m = rng.random(q)
        m[k] = 0.0
        if op(m) != 0.0:
            return False
    return True


def max_jump_at(op, m, k: int, eps: float = 1e-9) -> float:
    """|A(m with m_k = eps) - A(m with m_k = 0)|: a continuity probe."""
    a = np.array(m, dtype=float)
    b = a.copy()
    a[k], b[k] = 0.0, eps
    return abs(op(b) - op(a))
```

crtm/evaluation.py

SHA-256 0a79e58026043afadc2e276edcfc04efdbf7867c7eac01df9a2f768b48f6654f

```
"""Evaluation algorithm for one threat scenario (Section 6.7, Figure 8)
and roll-up to a resilience profile (Definition 10)."""
from __future__ import annotations

import math
from dataclasses import dataclass, field

import numpy as np

from . import aggregation as agg
from .metrics import Action
from .scale import level, level_abbr

OPERATORS = ("additive", "hard_gate", "power", "capped")


@dataclass
class ActionResult:
    ident: str
    goal: str
    status: str
    score: float
    level: int
    kappa: float
    lower: float
    upper: float
    contributions: list[float] = field(default_factory=list)
    normalized: list[float] = field(default_factory=list)
    inconsistent: list[int] = field(default_factory=list)

    def as_dict(self) -> dict:
        d = self.__dict__.copy()
        d["level_abbr"] = level_abbr(self.score)
        return d


def score_vector(m, w, K, operator: str = "hard_gate", gamma: float = 1.0, theta=0.5) -> float:
    """Apply one of the action operators to a complete vector."""
    if operator == "additive":
        return agg.additive(m, w)
    if operator == "hard_gate":
        return agg.hard_gate(m, w, K)
    if operator == "power":
        return agg.power(m, w, K, gamma)
    if operator == "capped":
        return agg.capped(m, w, K, theta)
    raise ValueError(f"unknown operator {operator!r}")


def evaluate_action(action: Action, method: str = "anchored", operator: str = "hard_gate",
                    gamma: float = 1.0, theta=0.5) -> ActionResult:
    """Score one basic action following Figure 8.

    1. validate class tables (8) and weights;
    2. refuse to score if a core (knockout) metric is missing;
    3. normalize each observed metric once;
    4. apply the operator; if metrics are missing, return mu_O with the
       enclosure of Proposition 6.
    """
    for mt in action.metrics:
        mt.table.validate()
    w = action.w()
    agg._w(w, action.q)
    K = action.core
    m = action.vector(method)
    inconsistent = [p for p, mt in enumerate(action.metrics)
                    if mt.observed and level(mt.normalized(method)) != mt.tabulated_level()]
    if any(math.isnan(m[k]) for k in K):
        return ActionResult(action.ident, action.goal, "not computed: missing core metric",
                            math.nan, 0, float(w[~np.isnan(m)].sum()), math.nan, math.nan,
                            normalized=m.tolist(), inconsistent=inconsistent)
    contrib = [float(w[p] * m[p]) if not math.isnan(m[p]) else math.nan for p in range(action.q)]
    if not np.isnan(m).any():
        s = score_vector(m, w, K, operator, gamma, theta)
        return ActionResult(action.ident, action.goal, "ok", s, level(s), 1.0, s, s,
                            contrib, m.tolist(), inconsistent)
    if operator not in ("hard_gate", "capped", "additive"):
        raise ValueError("the enclosure of Proposition 6 is stated for additive, gate and cap")
    enc = agg.missing_enclosure(m, w, K if operator != "additive" else (),
                                theta if operator == "capped" else None)
    return ActionResult(action.ident, action.goal, enc["status"], enc["mu_O"], level(enc["mu_O"]),
                        enc["kappa"], enc["lower"], enc["upper"], contrib, m.tolist(), inconsistent)


def evaluate_scenario(actions: dict[str, Action], selected=None, method: str = "anchored",
```

```
                      operator: str = "hard_gate", **kw) -> dict:
    """Evaluate the selected actions and roll up to goals (mean and minimum)."""
    selected = list(selected) if selected is not None else list(actions)
    res = {b: evaluate_action(actions[b], method, operator, **kw) for b in selected}
    goals: dict[str, list[float]] = {}
    for r in res.values():
        if not math.isnan(r.score):
            goals.setdefault(r.goal, []).append(r.score)
    profile = {g: {"mean": float(np.mean(v)), "min": float(np.min(v)),
                   "level_mean": level_abbr(np.mean(v)), "level_min": level_abbr(np.min(v))}
               for g, v in goals.items()}
    n_metrics = sum(actions[b].q for b in selected)
    return {"actions": res, "profile": profile, "N_theta": n_metrics,
            "observed": sum(mt.observed for b in selected for mt in actions[b].metrics)}


def dominates(p: dict, q: dict, key: str = "mean") -> bool:
    """Product order on profiles: p >= q in every goal."""
    return all(p[g][key] >= q[g][key] for g in q)
```

crtm/figures.py

SHA-256 ea1428af53194a91f5f6a77edb2642a4b78fffd05f7b1b7c460a083672b5dbb8

```
"""Figures of the digital supplement (matplotlib).

Every function returns a ``matplotlib.figure.Figure`` and draws only from
the data model, so the notebook and ``scripts/reproduce.py`` share one
source of truth.
"""
from __future__ import annotations

import math

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import BoundaryNorm, ListedColormap
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

from . import aggregation as agg
from . import sensitivity as sens
from . import uncertainty as unc
from .evaluation import evaluate_scenario
from .scale import LEVEL_ABBR, LEVEL_NAMES, Y, level

# five ordered levels: red arm -> neutral -> blue arm (diverging, gray midpoint)
LEVEL_COLORS = ("#c8433a", "#ef9a7f", "#e3e1dc", "#8fbbee", "#2a78d6")
LINEAR = "#eb6834"      # two-anchor linear normalization
ANCHORED = "#1f1f1d"    # anchored piecewise-linear normalization
MUTED = "#6b6a63"

plt.rcParams.update({
    "figure.dpi": 110, "savefig.dpi": 300, "font.size": 9,
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": "#8a8980", "axes.labelcolor": "#2b2b28",
    "xtick.color": "#4a4a45", "ytick.color": "#4a4a45",
    "axes.grid": False, "legend.frameon": False,
})


def _bands(ax, orientation: str = "h", alpha: float = 0.35):
    for r in range(5):
        if orientation == "h":
            ax.axhspan(Y[r], Y[r + 1], color=LEVEL_COLORS[r], alpha=alpha, lw=0, zorder=0)
        else:
            ax.axvspan(Y[r], Y[r + 1], color=LEVEL_COLORS[r], alpha=alpha, lw=0, zorder=0)


def _level_legend(ax, loc="lower center", ncol=5, **kw):
    handles = [Patch(color=c, alpha=0.6, label=n) for c, n in zip(LEVEL_COLORS, LEVEL_NAMES)]
    return ax.legend(handles=handles, loc=loc, ncol=ncol, fontsize=7.5, **kw)


# ---------------------------------------------------------------------- Fig. A
def fig_projections(scen: dict):
    """Action × technique incidence for the DDoS and phishing projections,
    with their meet and join (Proposition 2)."""
    P, Q = scen["projections"]["DDoS"], scen["projections"]["phishing"]
    J, M = P | Q, P & Q
    acts = sorted(J.actions, key=lambda b: (b[:2], b))
    techs = sorted({t for b in acts for t in J.techniques(b)})
    fig, ax = plt.subplots(figsize=(6.4, 4.6))
    for i, b in enumerate(acts):
        for j, t in enumerate(techs):
            inP, inQ = t in P.techniques(b), t in Q.techniques(b)
            if inP and inQ:
                ax.scatter(j, i, s=150, marker="s", color="#2a78d6", ec="white", lw=2, zorder=3)
            elif inP:
                ax.scatter(j, i, s=150, marker="s", color="#eb6834", ec="white", lw=2, zorder=3)
            elif inQ:
                ax.scatter(j, i, s=150, marker="s", color="#1baf7a", ec="white", lw=2, zorder=3)
    ax.set_xticks(range(len(techs)), techs)
    ax.set_yticks(range(len(acts)), [b + ("  ⊓" if b in M.actions else "") for b in acts])
    ax.invert_yaxis()
    ax.set_xlim(-0.6, len(techs) - 0.4)
    ax.tick_params(length=0)
    for s in ax.spines.values():
        s.set_visible(False)
    ax.set_title("Scenario projections: DDoS ⊔ phishing (|join| = %d, |meet| = %d actions)"
                 % (len(J.actions), len(M.actions)), fontsize=9, loc="left")
    ax.legend(handles=[Line2D([], [], ls="", marker="s", ms=9, color=c, label=l) for c, l in
                       (("#eb6834", "DDoS only"), ("#1baf7a", "phishing only"),
                        ("#2a78d6", "both (meet)"))],
              loc="upper left", bbox_to_anchor=(1.0, 1.0), fontsize=8)
    fig.tight_layout()
    return fig


```

```python
# ---------------------------------------------------------------- Fig. 5
def fig_normalizations(action):
    """Two-anchor linear vs anchored normalization for each metric of an action."""
    q = action.q
    cols = 3
    rows = math.ceil(q / cols)
    fig, axes = plt.subplots(rows, cols, figsize=(9.2, 2.9 * rows), squeeze=False)
    for p, (ax, mt) in enumerate(zip(axes.flat, action.metrics)):
        t = mt.table
        lo, hi = min(t.xi), max(t.xi)
        span = hi - lo
        xs = np.linspace(lo - 0.08 * span, hi + 0.08 * span, 1500)
        _bands(ax)
        ax.plot(xs, t.linear(xs), color=LINEAR, ls="--", lw=1.8, label="linear $n_{c,h}$")
        ax.plot(xs, t.anchored(xs), color=ANCHORED, lw=1.8, label="anchored $\\nu_\\xi$")
        # physical classes as a strip under the axis
        edges = [-np.inf, *t.eta[1:5], np.inf]
        for r in range(5):
            a, b = edges[r], edges[r + 1]
            xa, xb = sorted((t.sigma * a, t.sigma * b))
            ax.axvspan(max(xa, xs[0]), min(xb, xs[-1]), ymin=0, ymax=0.035,
                       color=LEVEL_COLORS[r], lw=0)
        if mt.observed:
            ax.plot(mt.value, t.linear(mt.value), "s", ms=7, mfc="white", mec=LINEAR, mew=1.6)
            ax.plot(mt.value, t.anchored(mt.value), "D", ms=6, color=ANCHORED, mec="white")
        ax.set_ylim(-0.03, 1.03)
        ax.set_xlim(xs[0], xs[-1])
        ax.set_title(f"M{p + 1}{'*' if mt.core else ''}: {mt.name}", fontsize=8, loc="left")
        ax.set_xlabel(mt.unit, fontsize=8)
        ax.set_yticks(Y)
    for ax in axes.flat[q:]:
        ax.set_visible(False)
    axes[0, 0].legend(loc="center right", fontsize=7.5)
    fig.suptitle(f"Normalizations of the metrics of {action.ident}  (□ linear, ◆ anchored, at the example value
)",
                 fontsize=9.5, x=0.01, ha="left")
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------- Fig. 6
def fig_consistency(actions):
    """Normalized scores against the band of the tabulated class (all metrics)."""
    rows = [(b, p, mt) for b, a in actions.items() for p, mt in enumerate(a.metrics)]
    fig, ax = plt.subplots(figsize=(9.6, 3.8))
    for i, (b, p, mt) in enumerate(rows):
        if not mt.observed:
            ax.text(i, 0.5, "n.r.", ha="center", va="center", fontsize=6.5, color=MUTED, rotation=90)
            continue
        r = mt.tabulated_level()
        ax.add_patch(plt.Rectangle((i - 0.38, Y[r - 1]), 0.76, Y[r] - Y[r - 1],
                                   color=LEVEL_COLORS[r - 1], alpha=0.75, lw=0))
        lin, anc = mt.normalized("linear"), mt.normalized("anchored")
        bad = level(lin) != r
        ax.plot(i, lin, "o", ms=6, mfc="white" if not bad else "#c8433a",
                mec="#c8433a" if bad else LINEAR, mew=1.4, zorder=4)
        ax.plot(i, anc, "D", ms=4.5, color=ANCHORED, zorder=5)
        if mt.core:
            ax.text(i, 1.04, "K", ha="center", fontsize=7, weight="bold")
    # action separators and labels
    start = 0
    for b, a in actions.items():
        end = start + a.q
        ax.axvline(end - 0.5, color="#c9c8c0", lw=0.8)
        ax.text((start + end - 1) / 2, -0.06, b, ha="right", va="top", fontsize=7.5, rotation=30,
                transform=ax.transData)
        start = end
    ax.set_xlim(-0.6, len(rows) - 0.4)
    ax.set_ylim(-0.02, 1.08)
    ax.set_xticks([])
    ax.set_yticks(Y)
    ax.set_ylabel("normalized score")
    n_bad = sum(1 for _, _, mt in rows if mt.observed and level(mt.normalized("linear")) != mt.tabulated_level(
))
    n_obs = sum(mt.observed for _, _, mt in rows)
    ax.set_title(f"Consistency with tabulated classes: linear outside its band in {n_bad} of {n_obs} "
                 f"observed metrics, anchored in 0", fontsize=9, loc="left")
    ax.legend(handles=[Line2D([], [], ls="", marker="o", mfc="white", mec=LINEAR, label="linear, consistent"),
                       Line2D([], [], ls="", marker="o", color="#c8433a", label="linear, inconsistent"),
                       Line2D([], [], ls="", marker="D", color=ANCHORED, label="anchored")],
              loc="upper center", bbox_to_anchor=(0.5, -0.2), ncol=3, fontsize=7.5)
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------- Fig. 7
```

```python
def fig_operators(action, core_pos: int = 0, method: str = "linear"):
    """Response of the operators to the core metric, other metrics fixed."""
    w = action.w()
    K = action.core
    base = action.vector(method)
    t = np.linspace(0, 1, 801)

    def curve(op):
        out = []
        for v in t:
            m = base.copy()
            m[core_pos] = v
            out.append(op(m))
        return np.array(out)

    C = float(w @ base - w[core_pos] * base[core_pos])
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(9.2, 3.5), sharey=True)
    for ax in (a1, a2):
        _bands(ax)
    add = curve(lambda m: agg.additive(m, w))
    a1.plot(t, add, ls="--", color=MUTED, lw=1.6, label="additive $S_w$ (= gate for $m_1>0$)")
    a1.plot([0], [0], "o", color=ANCHORED, ms=6, zorder=5, clip_on=False)
    a1.plot([0], [C], "o", mfc="white", mec=ANCHORED, ms=6, zorder=5, clip_on=False,
            label="hard gate jump at $m_1=0$")
    for g, c in zip((0.5, 1, 2), ("#86b6ef", "#2a78d6", "#104281")):
        a1.plot(t, curve(lambda m: agg.power(m, w, K, g)), color=c, lw=1.8, label=f"power, γ = {g}")
    a1.set_title("(a) additive, hard gate, power discounting", fontsize=9, loc="left")
    for th, c in zip((0.25, 0.5, 1.0), ("#86b6ef", "#2a78d6", "#104281")):
        y = curve(lambda m: agg.capped(m, w, K, th))
        a2.plot(t, y, color=c, lw=1.8, label=f"capped, θ = {th}")
        kink = th * C / (1 - w[core_pos] * th)   # m1/th = C + w1*m1
        if kink <= 1:
            a2.plot([kink], [kink / th], "|", color=c, ms=12, mew=2)
    a2.plot(t, add, ls="--", color=MUTED, lw=1.2, label="additive")
    a2.set_title("(b) capped knockout operator", fontsize=9, loc="left")
    tab = action.metrics[core_pos].table
    att = np.arange(int(min(tab.xi)), int(max(tab.xi)) + 1)
    for ax in (a1, a2):
        ax.set_xlabel(f"normalized core metric $m_{core_pos + 1}$ ({method})")
        ax.legend(fontsize=7, loc="lower right")
        ax.set_xlim(0, 1)
        ax.set_ylim(0, 1.0)
        sec = ax.secondary_xaxis(-0.25)
        sec.set_xticks([float(tab.normalize(v, method)) for v in att], [str(v) for v in att])
        sec.set_xlabel(action.metrics[core_pos].unit, fontsize=8)
    a1.set_ylabel(f"score of {action.ident}")
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------------- Fig. 9
def fig_contributions(action):
    """Contributions w_q m_q of the metrics under both normalizations."""
    w = action.w()
    fig, axes = plt.subplots(1, 2, figsize=(9.2, 3.2), sharey=True)
    for ax, meth, title in zip(axes, ("linear", "anchored"),
                               ("(a) two-anchor linear", "(b) anchored piecewise-linear")):
        m = action.vector(meth)
        c = w * m
        left = 0.0
        for p, v in enumerate(c):
            ax.barh(0, v, left=left, height=0.6, color=LEVEL_COLORS[level(m[p]) - 1],
                    ec="white", lw=2, hatch="///" if p in action.core else None)
            if v > 0.05:
                ax.text(left + v / 2, 0, f"M{p + 1}\n{v:.3f}", ha="center", va="center", fontsize=7.5)
            left += v
        ax.axvline(left, color=ANCHORED, lw=1.4)
        ax.text(left - 0.01, 0.36, f"μ = {left:.4f} ({LEVEL_ABBR[level(left) - 1]})", ha="right", fontsize=8.5)
        for r in range(5):
            ax.axvspan(Y[r], Y[r + 1], ymin=0, ymax=0.06, color=LEVEL_COLORS[r], lw=0)
        ax.set_xlim(0, 1)
        ax.set_ylim(-0.5, 0.5)
        ax.set_yticks([])
        ax.set_xticks(Y)
        ax.set_title(title, fontsize=9, loc="left")
        ax.set_xlabel("cumulative contribution $\\sum w_q m_q$")
    fig.suptitle(f"Decomposition of the local score of {action.ident} (hatched: knockout metric; bar colour: le
↪ vel of $m_q$)",
                 fontsize=9.5, x=0.01, ha="left")
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------------- Fig. 10
def fig_sensitivity(action, p1: int = 2, p2: int = 3, r1=(0, 200), r2=(0, 260)):
    """Score as a function of two response times; level regions and difference."""
```

```
    cmap = ListedColormap(LEVEL_COLORS)
    norm = BoundaryNorm(Y, 5)
    fig, axes = plt.subplots(1, 3, figsize=(11, 3.6))
    Zs = {}
    for ax, meth, title in zip(axes[:2], ("linear", "anchored"), ("(a) linear", "(b) anchored")):
        xs, ys, Z = sens.grid(action, p1, p2, r1, r2, meth)
        Zs[meth] = Z
        ax.pcolormesh(xs, ys, Z, cmap=cmap, norm=norm, shading="auto", alpha=0.85, rasterized=True)
        ax.contour(xs, ys, Z, levels=np.arange(0.5, 1.0, 0.05), colors="#55544e", linewidths=0.4)
        ax.contour(xs, ys, Z, levels=[0.7, 0.9], colors=ANCHORED, linewidths=1.6)
        ax.set_title(f"{title}: {Z.min():.3f} … {Z.max():.3f}", fontsize=9, loc="left")
    D = Zs["anchored"] - Zs["linear"]
    lim = np.abs(D).max()
    im = axes[2].pcolormesh(xs, ys, D, cmap="RdBu", vmin=-lim, vmax=lim, shading="auto", rasterized=True)
    fig.colorbar(im, ax=axes[2], fraction=0.05, label="anchored − linear")
    axes[2].set_title(f"(c) difference, max |Δ| = {lim:.3f}", fontsize=9, loc="left")
    m1, m2 = action.metrics[p1], action.metrics[p2]
    for ax in axes:
        for v in m1.table.xi[1:5]:
            ax.axvline(v, ls=":", color=MUTED, lw=0.8)
        for v in m2.table.xi[1:5]:
            ax.axhline(v, ls=":", color=MUTED, lw=0.8)
        ax.plot(m1.value, m2.value, "*", ms=11, color="white", mec=ANCHORED)
        ax.set_xlabel(f"τ{p1 + 1}: {m1.name} ({m1.unit})", fontsize=7.5)
        ax.set_ylabel(f"τ{p2 + 1}: {m2.name} ({m2.unit})", fontsize=7.5)
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------------- Fig. 11
def fig_dirichlet(action, alphas=(1, 4), n: int = unc.N_SAMPLES, seed: int = unc.SEED):
    """Distribution of the score under random Dirichlet weights."""
    fig, axes = plt.subplots(1, len(alphas), figsize=(9.2, 3.2), sharey=True)
    for ax, a in zip(np.atleast_1d(axes), alphas):
        _bands(ax, "v", alpha=0.25)
        for meth, c in (("linear", LINEAR), ("anchored", ANCHORED)):
            m = action.vector(meth)
            s = unc.sample_scores(m, a, n, seed)
            mean, sd = unc.dirichlet_moments(m, a)
            ax.hist(s, bins=np.linspace(0.45, 1.0, 111), density=True, histtype="step", lw=1.6, color=c,
                    label=f"{meth}: P(high) = {np.mean(s >= 0.9):.1%}")
            ax.axvline(mean, color=c, ls=":", lw=1.2)
            ax.axvline(agg.additive(m, action.w()), color=c, ls="--", lw=1.2)
        ax.set_title(f"w ~ Dirichlet({a}·1)", fontsize=9, loc="left")
        ax.set_xlabel(f"score of {action.ident}")
        ax.legend(fontsize=7.5, loc="upper left")
        ax.set_xlim(0.45, 1.0)
    np.atleast_1d(axes)[0].set_ylabel("density")
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------------- Fig. 12
def fig_profile(actions):
    """Action scores (both normalizations, enclosure) and goal roll-ups."""
    res = {m: evaluate_scenario(actions, method=m) for m in ("linear", "anchored")}
    names = list(actions)
    fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 3.8), gridspec_kw={"width_ratios": [3, 2]})
    _bands(a1)
    _bands(a2)
    x = np.arange(len(names))
    for dx, meth, c, mk in ((-0.12, "linear", LINEAR, "o"), (0.12, "anchored", ANCHORED, "D")):
        r = res[meth]["actions"]
        s = [r[b].score for b in names]
        lo = [r[b].score - r[b].lower for b in names]
        hi = [r[b].upper - r[b].score for b in names]
        a1.errorbar(x + dx, s, yerr=[lo, hi], fmt=mk, color=c, ms=6, capsize=3, label=meth)
    a1.set_xticks(x, [f"{b}\n{actions[b].goal}" for b in names], fontsize=7.5)
    a1.set_ylim(0.4, 1.0)
    a1.set_yticks(Y[2:])
    a1.set_ylabel("action score (hard gate)")
    a1.set_title("(a) action scores; bars = enclosure for missing metrics", fontsize=9, loc="left")
    a1.legend(fontsize=7.5, loc="upper right")
    goals = list(res["anchored"]["profile"])
    gx = np.arange(len(goals))
    for dx, meth, c in ((-0.15, "linear", LINEAR), (0.15, "anchored", ANCHORED)):
        pr = res[meth]["profile"]
        a2.bar(gx + dx, [pr[g]["mean"] - 0.4 for g in goals], bottom=0.4, width=0.28, color=c,
               alpha=0.85, ec="white", lw=2, label=f"{meth}: mean")
        a2.plot(gx + dx, [pr[g]["min"] for g in goals], "_", ms=14, mew=2.5, color="#c8433a")
    a2.plot([], [], "_", ms=12, mew=2.5, color="#c8433a", label="minimum roll-up")
    a2.set_xticks(gx, goals)
    a2.set_ylim(0.4, 1.0)
    a2.set_yticks(Y[2:])
    a2.set_title("(b) resilience profile π (goal roll-ups)", fontsize=9, loc="left")
    a2.legend(fontsize=7.5, loc="upper right")
```

```
    fig.tight_layout()
    return fig


ALL = {
    "FigS_A_projections": ("scen", fig_projections),
    "FigS_05_normalizations_UN.S1.A1": ("UN.S1.A1", fig_normalizations),
    "FigS_06_consistency": ("actions", fig_consistency),
    "FigS_07_operators_UN.S1.A1": ("UN.S1.A1", fig_operators),
    "FigS_09_contributions_UN.S1.A1": ("UN.S1.A1", fig_contributions),
    "FigS_10_sensitivity_UN.S1.A1": ("UN.S1.A1", fig_sensitivity),
    "FigS_11_dirichlet_UN.S1.A1": ("UN.S1.A1", fig_dirichlet),
    "FigS_12_profile_phishing": ("actions", fig_profile),
}
```

crtm/hierarchy.py

SHA-256 5b7a63c7ec41384a3823f356bf7d8b6e37707bceba21a102e28f348e0478cfae

```
"""Set-theoretic data model (Section 3).

  * identifier map iota(i, j, k) = c_i.Sj.Ak and its inverse (Proposition 1);
  * technique alphabet with an injective coding (Definition 2);
  * scenario projections, admissibility (3), order, join and meet (4)
    and the down-set encoding of Proposition 2;
  * Jaccard distance between projections (Section 3.3).
"""
from __future__ import annotations

import re
import unicodedata
from dataclasses import dataclass, field
from typing import Iterable, Mapping

#: CREF tasks (objectives) in index order i = 1..8
TASKS = {1: ("PA", "Prevent/Avoid"), 2: ("PR", "Prepare"), 3: ("CN", "Continue"),
         4: ("CS", "Constrain"), 5: ("RE", "Reconstitute"), 6: ("UN", "Understand"),
         7: ("TR", "Transform"), 8: ("RA", "Re-Architect")}

#: CREF goals
GOALS = {"ANT": "Anticipate", "WITH": "Withstand", "REC": "Recover", "AD": "Adapt"}

#: the fourteen techniques of NIST SP 800-160 Vol. 2 with disambiguated codes
TECHNIQUES = {"AR": "Adaptive Response", "AM": "Analytic Monitoring",
              "CA": "Contextual Awareness", "CP": "Coordinated Protection",
              "DE": "Deception", "DV": "Diversity", "DP": "Dynamic Positioning",
              "NP": "Non-Persistence", "PR": "Privilege Restriction",
              "RD": "Redundancy", "RL": "Realignment", "S": "Segmentation",
              "SI": "Substantiated Integrity", "U": "Unpredictability"}

_CODE_RE = re.compile(r"^[A-Z]+$")
_ID_RE = re.compile(r"^([A-Z]+)\.S([1-9][0-9]*)\.A([1-9][0-9]*)$")
_TASK_BY_CODE = {c: i for i, (c, _) in TASKS.items()}


# ---------------------------------------------------------------- identifiers
def check_code(code: str) -> list[str]:
    """Return problems with a task/technique code (empty list if valid).

    Detects non-Latin look-alike letters, e.g. a Cyrillic 'C' in 'CN'
    (one such entry was found in the source tabulation, Section 3.4).
    """
    problems = []
    for ch in code:
        if not ("A" <= ch <= "Z"):
            name = unicodedata.name(ch, "UNKNOWN")
            problems.append(f"character {ch!r} (U+{ord(ch):04X}, {name}) is not a Latin capital")
    if not code:
        problems.append("empty code")
    return problems


def iota(i: int, j: int, k: int, tasks: Mapping[int, tuple[str, str]] = TASKS) -> str:
    """Identifier map iota(i, j, k) = c_i.Sj.Ak (Definition 1)."""
    if min(i, j, k) < 1:
        raise ValueError("indices start at 1")
    return f"{tasks[i][0]}.S{j}.A{k}"


def iota_inv(identifier: str, tasks: Mapping[int, tuple[str, str]] = TASKS) -> tuple[int, int, int]:
    """Inverse of iota by parsing (Proposition 1)."""
    mt = _ID_RE.match(identifier)
    if not mt:
        bad = check_code(identifier.split(".")[0])
        raise ValueError(f"malformed identifier {identifier!r}" + (f": {bad}" if bad else ""))
    by_code = {c: i for i, (c, _) in tasks.items()}
    code, j, k = mt.group(1), int(mt.group(2)), int(mt.group(3))
    if code not in by_code:
        raise ValueError(f"unknown task code {code!r}")
    return by_code[code], j, k


def is_injective(mapping: Mapping) -> bool:
    """True if no two keys share a value."""
    vals = list(mapping.values())
    return len(vals) == len(set(vals))


# ---------------------------------------------------------------- projections
@dataclass(frozen=True)
class Projection:
    """Projection P = (I_P, alpha_P) of the data model (Definition 3)."""

    alpha: Mapping[str, frozenset] = field(default_factory=dict)
```

```
    name: str = ""

    def __post_init__(self):
        object.__setattr__(self, "alpha",
                           {b: frozenset(a) for b, a in dict(self.alpha).items()})

    @property
    def actions(self) -> frozenset:
        return frozenset(self.alpha)

    def techniques(self, beta: str) -> frozenset:
        return self.alpha.get(beta, frozenset())

    # admissibility (3) ------------------------------------------------
    def violations(self, catalog: Mapping[str, Iterable[str]]) -> list[str]:
        out = []
        for b, a in self.alpha.items():
            if b not in catalog:
                out.append(f"{b}: not in the catalog index set")
            elif not a <= frozenset(catalog[b]):
                extra = sorted(a - frozenset(catalog[b]))
                out.append(f"{b}: techniques {extra} not in catalog set {sorted(catalog[b])}")
        return out

    def is_admissible(self, catalog: Mapping[str, Iterable[str]]) -> bool:
        return not self.violations(catalog)

    # order and lattice operations (4) ---------------------------------
    def __le__(self, other: "Projection") -> bool:
        return all(b in other.alpha and a <= other.alpha[b] for b, a in self.alpha.items())

    def __or__(self, other: "Projection") -> "Projection":   # join
        keys = self.actions | other.actions
        return Projection({b: self.techniques(b) | other.techniques(b) for b in keys},
                          name=f"({self.name} ⊔ {other.name})")

    def __and__(self, other: "Projection") -> "Projection":  # meet
        keys = self.actions & other.actions
        return Projection({b: self.techniques(b) & other.techniques(b) for b in keys},
                          name=f"({self.name} ⊓ {other.name})")

    def __eq__(self, other) -> bool:
        return isinstance(other, Projection) and dict(self.alpha) == dict(other.alpha)

    def __hash__(self):
        return hash(frozenset(self.alpha.items()))

    # down-set encoding (proof of Proposition 2) -----------------------
    def downset(self) -> frozenset:
        """E(P) = {(beta, *)} ∪ {(beta, a) : a in alpha_P(beta)}."""
        return frozenset([(b, "*") for b in self.alpha] +
                         [(b, a) for b, s in self.alpha.items() for a in s])


def poset_size(catalog: Mapping[str, Iterable[str]]) -> int:
    """|U| = |I| + sum |alpha(beta)| (Proposition 2)."""
    return len(catalog) + sum(len(set(a)) for a in catalog.values())


def jaccard_index(a: frozenset, b: frozenset) -> float:
    union = a | b
    return 1.0 if not union else len(a & b) / len(union)


def jaccard_distance(P: Projection, Q: Projection, on: str = "actions") -> float:
    """d_J on action sets (on='actions') or on down-sets (on='downset')."""
    if on == "actions":
        return 1.0 - jaccard_index(P.actions, Q.actions)
    if on == "downset":
        return 1.0 - jaccard_index(P.downset(), Q.downset())
    raise ValueError("on must be 'actions' or 'downset'")
```

crtm/metrics.py

SHA-256 5961d698e7d33c8a2a78a1ded18ff857508816f22963e76a96175864c2e6bfd5

```
"""Metric model (Section 4) and loaders for the supplementary data files."""
from __future__ import annotations

import csv
import json
import math
from dataclasses import dataclass, field
from pathlib import Path

import numpy as np

from .hierarchy import Projection
from .normalization import ClassTable

DATA_DIR = Path(__file__).resolve().parent.parent / "data"


@dataclass
class Metric:
    """Metric descriptor M = (X, u, sigma, tau, A, xi) with an observed value (Definition 4)."""

    code: str
    name: str
    unit: str
    table: ClassTable
    value: float = math.nan
    core: bool = False
    timing: str = ""            # 'ex-ante', 'ex-post' or '' (unspecified, ⊥)
    printed_classes: str = ""
    support: frozenset = frozenset()

    @property
    def observed(self) -> bool:
        return not math.isnan(self.value)

    @property
    def sigma(self) -> int:
        return self.table.sigma

    def normalized(self, method: str = "anchored") -> float:
        return math.nan if not self.observed else float(self.table.normalize(self.value, method))

    def tabulated_level(self) -> int:
        return 0 if not self.observed else int(self.table.tabulated_level(self.value))


@dataclass
class Action:
    """Basic action beta with its ordered metric tuple M(beta) and weights."""

    ident: str
    goal: str
    metrics: list[Metric] = field(default_factory=list)
    weights: np.ndarray | None = None
    weights_specified: bool = False

    @property
    def q(self) -> int:
        return len(self.metrics)

    @property
    def core(self) -> list[int]:
        return [p for p, m in enumerate(self.metrics) if m.core]

    def w(self) -> np.ndarray:
        if self.weights is None:
            return np.full(self.q, 1.0 / self.q)
        return np.asarray(self.weights, dtype=float)

    def vector(self, method: str = "anchored") -> np.ndarray:
        return np.array([m.normalized(method) for m in self.metrics])

    def check_support(self, alpha: frozenset) -> list[str]:
        """Support condition A_p ⊆ alpha(beta) of Definition 4."""
        return [f"{self.ident} p={p + 1}: support {sorted(m.support)} ⊄ {sorted(alpha)}"
                for p, m in enumerate(self.metrics) if not m.support <= alpha]


def _f(s: str) -> float:
    return math.nan if s.strip() == "" else float(s)


def load_metrics(path: str | Path | None = None) -> dict[str, Action]:
    """Load ``phishing_metrics.csv`` into an ordered dict of actions."""
    path = Path(path) if path else DATA_DIR / "phishing_metrics.csv"
```

```
    actions: dict[str, Action] = {}
    raw_w: dict[str, list[float]] = {}
    with open(path, encoding="utf-8", newline="") as fh:
        for row in csv.DictReader(fh):
            a = actions.setdefault(row["action"], Action(row["action"], row["goal"]))
            xi = tuple(float(row[f"xi{r}"]) for r in range(6))
            a.metrics.append(Metric(code=row["code"], name=row["metric"], unit=row["unit"],
                                    table=ClassTable(xi), value=_f(row["value"]),
                                    core=row["core"].strip() == "1", timing=row["timing"],
                                    printed_classes=row["printed_classes"]))
            raw_w.setdefault(a.ident, []).append(_f(row["weight"]))
    for a in actions.values():
        w = raw_w[a.ident]
        if all(not math.isnan(v) for v in w):
            a.weights, a.weights_specified = np.array(w), True
            if not math.isclose(sum(w), 1.0, abs_tol=1e-9):
                raise ValueError(f"weights of {a.ident} do not sum to 1")
    return actions


def load_scenarios(path: str | Path | None = None) -> dict:
    """Load ``scenarios.json``: catalog, projections and goal assignment."""
    path = Path(path) if path else DATA_DIR / "scenarios.json"
    with open(path, encoding="utf-8") as fh:
        d = json.load(fh)
    d["projections"] = {k: Projection(v["actions"], name=k) for k, v in d["scenarios"].items()}
    return d
```

crtm/normalization.py

SHA-256 f186802d42fac919b82b897f5f0a4726d36428c321744928b954579872bb8300

```
"""Class tables and level-consistent normalization (Section 5).

Implements
  * Definition 6  – class table xi, quality anchors eta = sigma * xi,
                    tabulated level lambda_xi (eq. 9);
  * eq. (10)      – two-anchor linear normalization n_{c,h};
  * Theorem 1     – existence test for a consistent clipped affine map;
  * Corollary 1   – interval of class-4 values sent to the band "high";
  * Definition 7  – anchored piecewise-linear normalization nu_xi (eq. 12);
  * a numerical grid check of consistency (Section 7.1).
"""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Callable, Sequence

import numpy as np

from .scale import Y, level


class ClassTableError(ValueError):
    """Raised when a class table violates the ordering condition (8)."""


@dataclass(frozen=True)
class ClassTable:
    """Class table xi in R^6 of a metric (Definition 6).

    Parameters
    ----------
    xi : six anchors (xi_0, ..., xi_5) in physical units.
    sigma : polarity, +1 if larger values mean more resilience, -1 otherwise.
        If omitted it is inferred from the order of xi_0 and xi_4.
    """

    xi: tuple[float, ...]
    sigma: int | None = None
    eta: np.ndarray = field(init=False, repr=False, compare=False)

    def __post_init__(self):
        xi = tuple(float(v) for v in self.xi)
        if len(xi) != 6:
            raise ClassTableError("a class table has exactly six anchors")
        sigma = self.sigma
        if sigma is None:
            sigma = 1 if xi[4] > xi[0] else -1
        if sigma not in (1, -1):
            raise ClassTableError("polarity must be +1 or -1")
        eta = sigma * np.asarray(xi)
        object.__setattr__(self, "xi", xi)
        object.__setattr__(self, "sigma", int(sigma))
        object.__setattr__(self, "eta", eta)
        self.validate()

    # ------------------------------------------------------------------ checks
    def validate(self) -> None:
        """Check condition (8): eta0 <= eta1 < eta2 < eta3 < eta4 <= eta5."""
        e = self.eta
        ok = e[0] <= e[1] < e[2] < e[3] < e[4] <= e[5]
        if not ok:
            raise ClassTableError(
                f"anchors violate (8): eta = {tuple(np.round(e, 6))}")

    # ------------------------------------------------------------ tabulated level
    def tabulated_level(self, x):
        """Level lambda_xi(x) that the class table assigns to x (eq. 9)."""
        u = self.sigma * np.asarray(x, dtype=float)
        e = self.eta
        r = np.searchsorted(e[1:5], u, side="right") + 1
        r = np.where(np.isnan(u), 0, r)
        return int(r) if np.ndim(r) == 0 else r.astype(int)

    # ------------------------------------------------------------- normalizations
    def linear(self, x):
        """Two-anchor linear normalization n_{c,h} with c = xi_0, h = xi_4 (eq. 10)."""
        c, h = self.xi[0], self.xi[4]
        out = np.clip((np.asarray(x, dtype=float) - c) / (h - c), 0.0, 1.0)
        return float(out) if out.ndim == 0 else out

    def anchored(self, x):
        """Anchored piecewise-linear normalization nu_xi (Definition 7, eq. 12).

        Degenerate segments (eta_r = eta_{r+1}, possible for r = 0 and r = 4)
        are skipped. At a degenerate critical anchor eta_0 = eta_1 the value
```

```
        at u = eta_1 is y_1, so that nu_xi is consistent with xi on all of R
        (Theorem 2a).
        """
        u = self.sigma * np.atleast_1d(np.asarray(x, dtype=float))
        e = self.eta
        out = np.full(u.shape, np.nan)
        out[u <= e[0]] = 0.0
        for r in range(5):
            if e[r] < e[r + 1]:
                m = (u >= e[r]) & (u < e[r + 1])
                out[m] = Y[r] + (Y[r + 1] - Y[r]) * (u[m] - e[r]) / (e[r + 1] - e[r])
        out[u >= e[5]] = 1.0
        if e[4] == e[5]:
            out[u >= e[4]] = 1.0
        out[np.isnan(u)] = np.nan
        return float(out[0]) if np.ndim(x) == 0 else out

    def normalize(self, x, method: str = "anchored"):
        """Dispatch to ``linear`` or ``anchored``."""
        if method == "linear":
            return self.linear(x)
        if method == "anchored":
            return self.anchored(x)
        raise ValueError(f"unknown normalization {method!r}")

    # ------------------------------------------------------------ theory checks
    def affine_consistent(self, rtol: float = 1e-9) -> tuple[bool, float | None, float | None]:
        """Theorem 1: does a consistent clipped affine map a*u + b exist?

        Returns (exists, a, b). Condition (11): equal consecutive slopes
        (eta_{r+1} - eta_r) / (y_{r+1} - y_r) for r = 1, 2, 3.
        """
        e = self.eta
        slopes = [(e[r + 1] - e[r]) / (Y[r + 1] - Y[r]) for r in (1, 2, 3)]
        exists = bool(np.allclose(slopes, slopes[0], rtol=rtol, atol=0.0))
        if not exists:
            return False, None, None
        a = 1.0 / slopes[0]
        b = Y[1] - a * e[1]
        return True, float(a), float(b)

    def corollary1_interval(self) -> dict:
        """Corollary 1: class-4 values that n_{c,h} maps into the band 'high'.

        Returns the interval in quality coordinates u = sigma*x, the same
        interval in physical units and its length
        min{eta4 - eta3, (1 - y4)(eta4 - eta0)}.
        """
        e = self.eta
        lo = max(e[3], e[0] + Y[4] * (e[4] - e[0]))
        hi = e[4]
        length = min(e[4] - e[3], (1 - Y[4]) * (e[4] - e[0]))
        if self.sigma == 1:
            phys = {"from": float(lo), "to": float(hi), "closed": "[from, to)"}
        else:
            phys = {"from": float(-hi), "to": float(-lo), "closed": "(from, to]"}
        return {"u_interval": (float(lo), float(hi)), "x_interval": phys,
                "length": float(length)}

    def anchors_with_neighbours(self, eps: float = 1e-9) -> np.ndarray:
        """Physical anchors and points within eps of them (for grid checks)."""
        pts = []
        for v in self.xi:
            pts += [v - eps, v, v + eps]
        return np.asarray(pts)


# ------------------------------------------------------------------ utilities
def is_consistent(table: ClassTable, norm: Callable, xs: Sequence[float]) -> np.ndarray:
    """Boolean array: lambda(n(x)) == lambda_xi(x) for each x in xs."""
    xs = np.asarray(xs, dtype=float)
    return level(norm(xs)) == table.tabulated_level(xs)


def grid_check(table: ClassTable, method: str = "anchored", n: int = 20001,
               margin: float = 0.25, eps: float = 1e-9) -> dict:
    """Numerical consistency check on a dense grid plus all anchors ± eps.

    The grid covers [min xi - margin*span, max xi + margin*span].
    """
    lo, hi = min(table.xi), max(table.xi)
    span = hi - lo
    xs = np.concatenate([np.linspace(lo - margin * span, hi + margin * span, n),
                         table.anchors_with_neighbours(eps)])
    ok = is_consistent(table, lambda v: table.normalize(v, method), xs)
    return {"points": int(xs.size), "violations": int((~ok).sum()),
```

```
172             "worst_x": xs[~ok].tolist()[:10]}
```

crtm/scale.py

SHA-256 508915894eeb1f71f1feac4bdee965d73915ee5ca0ef0b0a646c9363515a9f83

```
"""Five-level linguistic scale (Definition 5).

Score breakpoints y = (0, 0.2, 0.4, 0.7, 0.9, 1) define the bands
Y_r = [y_{r-1}, y_r) for r = 1..4 and Y_5 = [y_4, y_5], named
critical, below average, average, above average and high.
"""
from __future__ import annotations

import numpy as np

#: score breakpoints y_0 .. y_5
Y = np.array([0.0, 0.2, 0.4, 0.7, 0.9, 1.0])

#: level names, index r - 1 for r = 1..5
LEVEL_NAMES = ("critical", "below average", "average", "above average", "high")

#: abbreviations used in the article tables
LEVEL_ABBR = ("C", "BA", "A", "AA", "H")


def level(s):
    """Level map lambda: [0, 1] -> {1, ..., 5} (vectorised).

    Returns an int (scalar input) or an int array. NaN maps to 0
    ("not computed").
    """
    s_arr = np.asarray(s, dtype=float)
    r = np.searchsorted(Y[1:5], s_arr, side="right") + 1
    r = np.where(np.isnan(s_arr), 0, r)
    if r.ndim == 0:
        return int(r)
    return r.astype(int)


def level_abbr(s) -> str:
    """Abbreviated level name of a single score, '—' for NaN."""
    r = level(s)
    return "—" if r == 0 else LEVEL_ABBR[r - 1]


def level_name(s) -> str:
    """Full level name of a single score."""
    r = level(s)
    return "not computed" if r == 0 else LEVEL_NAMES[r - 1]


def band(r: int) -> tuple[float, float]:
    """Score band (y_{r-1}, y_r) of level r."""
    if not 1 <= r <= 5:
        raise ValueError("level must be in 1..5")
    return float(Y[r - 1]), float(Y[r])
```

crtm/sensitivity.py

SHA-256 70d56785106a40f79d211a544dac49e05548a58a56ca65a61a1dd7604bb3c060

```
"""Sensitivity of an action score to two response times (Section 7.4)."""
from __future__ import annotations

import numpy as np
from scipy.optimize import brentq

from . import aggregation as agg
from .metrics import Action


def score_with(action: Action, values: dict[int, float], method: str) -> float:
    """Hard-gated score of ``action`` with metric values replaced (0-based positions)."""
    m = []
    for p, mt in enumerate(action.metrics):
        x = values.get(p, mt.value)
        m.append(float(mt.table.normalize(x, method)))
    return agg.hard_gate(np.array(m), action.w(), action.core)


def grid(action: Action, p1: int, p2: int, r1, r2, method: str, n1: int = 401, n2: int = 521):
    """Score on a (n2 x n1) grid over metric p1 (x-axis) and p2 (y-axis)."""
    xs, ys = np.linspace(*r1, n1), np.linspace(*r2, n2)
    t1 = action.metrics[p1].table.normalize(xs, method)
    t2 = action.metrics[p2].table.normalize(ys, method)
    base = action.vector(method)
    w = action.w()
    rest = base @ w - w[p1] * base[p1] - w[p2] * base[p2]
    Z = rest + w[p1] * t1[None, :] + w[p2] * t2[:, None]
    return xs, ys, Z


def threshold(action: Action, p: int, target: float, method: str, lo: float, hi: float,
              fixed: dict[int, float] | None = None) -> float:
    """Value of metric p at which the score crosses ``target`` (root in [lo, hi])."""
    fixed = dict(fixed or {})
    f = lambda x: score_with(action, {**fixed, p: x}, method) - target
    return float(brentq(f, lo, hi, xtol=1e-10))
```

crtm/uncertainty.py

SHA-256 c5945d915d6c9443c7b3e178f46f317b97e8cd63439ee1524c78e9966d982750

```
"""Uncertainty in the weights (Section 6.5, Proposition 7)."""
from __future__ import annotations

import numpy as np

from .scale import Y, level

SEED = 20261027          # fixed seed used in the article
N_SAMPLES = 200_000      # weight vectors per configuration


def dirichlet_moments(m, alpha: float) -> tuple[float, float]:
    """Exact mean and standard deviation of S_w(m) for w ~ Dirichlet(alpha 1_q) (eq. 19)."""
    m = np.asarray(m, dtype=float)
    q = m.size
    s2 = np.mean((m - m.mean()) ** 2)
    return float(m.mean()), float(np.sqrt(s2 / (q * alpha + 1)))


def sample_scores(m, alpha: float, n: int = N_SAMPLES, seed: int = SEED) -> np.ndarray:
    """Monte Carlo sample of S_w(m) under symmetric Dirichlet weights."""
    m = np.asarray(m, dtype=float)
    rng = np.random.default_rng(seed)
    w = rng.dirichlet(np.full(m.size, alpha), size=n)
    return w @ m


def level_probabilities(scores) -> dict[int, float]:
    """Empirical Pr[lambda(S) = r], r = 1..5."""
    lv = level(scores)
    return {r: float(np.mean(lv == r)) for r in range(1, 6)}


def summary(m, alpha: float, n: int = N_SAMPLES, seed: int = SEED) -> dict:
    s = sample_scores(m, alpha, n, seed)
    mean, sd = dirichlet_moments(m, alpha)
    return {"alpha": alpha, "exact_mean": mean, "exact_sd": sd,
            "mc_mean": float(s.mean()), "mc_sd": float(s.std()),
            "p_high": float(np.mean(s >= Y[4])), "p_below_0.7": float(np.mean(s < Y[3])),
            "levels": level_probabilities(s)}
```

scripts/reproduce.py

SHA-256 57ecadde9ee75820d2df3069524dd14f728cf2fa66a689acf6974d0512c360bd

```
#!/usr/bin/env python3
"""Reproduce every number, table and figure of the illustrative application.

Usage:  python scripts/reproduce.py [--out results] [--no-figures] [--fast]

Writes
  results/results.json            all computed quantities (Data S1)
  results/checks.csv              computed vs published values
  results/tables/*.csv            Tables 5, 6 and A1
  results/figures/*.{png,svg,pdf} supplementary figures
and exits with status 1 if any published value is not reproduced.
"""
from __future__ import annotations

import argparse
import csv
import json
import math
import sys
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

import crtm  # noqa: E402
from crtm import aggregation as agg  # noqa: E402
from crtm import sensitivity as sens  # noqa: E402
from crtm import uncertainty as unc  # noqa: E402
from crtm.hierarchy import jaccard_index, poset_size  # noqa: E402
from crtm.normalization import grid_check  # noqa: E402

CHECKS: list[dict] = []


def check(name: str, computed, published, tol: float, where: str = ""):
    """Record a comparison with a value printed in the article."""
    ok = abs(float(computed) - float(published)) <= tol
    CHECKS.append({"quantity": name, "computed": float(computed), "published": float(published),
                   "tolerance": tol, "ok": ok, "where": where})
    return ok


def jsonable(o):
    if isinstance(o, dict):
        return {str(k): jsonable(v) for k, v in o.items()}
    if isinstance(o, (list, tuple)):
        return [jsonable(v) for v in o]
    if isinstance(o, (np.floating, float)):
        return None if math.isnan(o) else float(o)
    if isinstance(o, (np.integer,)):
        return int(o)
    if isinstance(o, np.ndarray):
        return jsonable(o.tolist())
    return o


def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--out", default=str(ROOT / "results"))
    ap.add_argument("--no-figures", action="store_true")
    ap.add_argument("--fast", action="store_true", help="20,000 instead of 200,000 Monte Carlo samples")
    args = ap.parse_args(argv)
    out = Path(args.out)
    (out / "tables").mkdir(parents=True, exist_ok=True)
    n_mc = 20_000 if args.fast else unc.N_SAMPLES
    mc_tol = 0.01 if args.fast else 0.003

    actions = crtm.load_metrics()
    scen = crtm.load_scenarios()
    R: dict = {"version": crtm.__version__, "seed": unc.SEED, "mc_samples": n_mc}

    # ---------------------------------------------------- Section 3.5: lattice
    cat = scen["catalog"]
    P, Q = scen["projections"]["DDoS"], scen["projections"]["phishing"]
    J, M = P | Q, P & Q
    R["projections"] = {
        "admissible": {k: v.is_admissible(cat) for k, v in scen["projections"].items()},
        "sizes": {"DDoS": len(P.actions), "phishing": len(Q.actions)},
        "meet": sorted(M.actions), "join_size": len(J.actions),
        "jaccard_actions": jaccard_index(P.actions, Q.actions),
        "downset_sizes": [len(P.downset()), len(Q.downset())],
        "downset_meet": len(P.downset() & Q.downset()), "downset_join": len(P.downset() | Q.downset()),
        "jaccard_downset": jaccard_index(P.downset(), Q.downset()),
```

```
        "poset_size_local_catalog": poset_size(cat),
    }
    pr = R["projections"]
    check("|P_DDoS|", pr["sizes"]["DDoS"], 9, 0, "Sec. 3.5")
    check("|P_phish|", pr["sizes"]["phishing"], 7, 0, "Sec. 3.5")
    check("|meet|", len(pr["meet"]), 4, 0, "Sec. 3.5")
    check("|join|", pr["join_size"], 12, 0, "Sec. 3.5")
    check("Jaccard index (actions)", pr["jaccard_actions"], 1 / 3, 1e-12, "Sec. 3.5")
    check("|E(P_DDoS)|", pr["downset_sizes"][0], 26, 0, "Sec. 3.5")
    check("|E(P_phish)|", pr["downset_sizes"][1], 23, 0, "Sec. 3.5")
    check("|E meet|", pr["downset_meet"], 13, 0, "Sec. 3.5")
    check("|E join|", pr["downset_join"], 36, 0, "Sec. 3.5")
    check("Jaccard index (down-sets)", pr["jaccard_downset"], 0.361, 5e-4, "Sec. 3.5")

    # ------------------------------------------ Section 5: normalization theory
    un = actions["UN.S1.A1"]
    t3 = un.metrics[2].table
    c1 = t3.corollary1_interval()
    R["corollary1_UN.S1.A1_M3"] = c1
    check("Cor.1 interval length (M3)", c1["length"], 17, 1e-9, "Sec. 5.2")
    check("Cor.1 upper end (M3, min)", c1["x_interval"]["to"], 27, 1e-9, "Sec. 5.2")
    check("n_ch(4 sharing events)", un.metrics[4].normalized("linear"), 0.444, 5e-4, "Sec. 5.2")
    R["affine_consistent_exists"] = {f"{b}/M{p + 1}": mt.table.affine_consistent()[0]
                                     for b, a in actions.items() for p, mt in enumerate(a.metrics)}

    # consistency on all observed metrics (Fig. 6)
    bad_lin = bad_anc = n_obs = 0
    per_action = {}
    for b, a in actions.items():
        k = 0
        for mt in a.metrics:
            if not mt.observed:
                continue
            n_obs += 1
            r = mt.tabulated_level()
            if crtm.level(mt.normalized("linear")) != r:
                bad_lin += 1
                k += 1
            if crtm.level(mt.normalized("anchored")) != r:
                bad_anc += 1
        per_action[b] = k
    R["consistency"] = {"observed": n_obs, "inconsistent_linear": bad_lin,
                        "inconsistent_anchored": bad_anc, "per_action_linear": per_action}
    check("observed metrics", n_obs, 38, 0, "Sec. 5.3 / Fig. 6")
    check("inconsistent linear scores", bad_lin, 9, 0, "Abstract / Fig. 6")
    check("inconsistent anchored scores", bad_anc, 0, 0, "Theorem 2(a)")
    for b, v in zip(actions, (2, 1, 2, 2, 2, 0, 0)):
        check(f"inconsistent linear in {b}", per_action[b], v, 0, "Table 6")

    # grid check (Sec. 7.1)
    grid = [grid_check(mt.table, "anchored") for a in actions.values() for mt in a.metrics]
    R["grid_check"] = {"tables": len(grid), "points_per_table": grid[0]["points"],
                       "violations": sum(g["violations"] for g in grid)}
    check("grid-check violations (anchored)", R["grid_check"]["violations"], 0, 0, "Sec. 7.1")

    # ------------------------------------------------------ Table 5 / Sec. 7.2
    w = un.w()
    rows5 = []
    for p, mt in enumerate(un.metrics):
        rows5.append({"q": p + 1, "metric": mt.name, "unit": mt.unit, "value": mt.value,
                      "xi": " ".join(f"{v:g}" for v in mt.table.xi),
                      "n_ch": mt.normalized("linear"), "lvl_lin": crtm.level_abbr(mt.normalized("linear")),
                      "nu": mt.normalized("anchored"), "lvl_anc": crtm.level_abbr(mt.normalized("anchored")),
                      "w": w[p], "w_n": w[p] * mt.normalized("linear"), "w_nu": w[p] * mt.normalized("anchored"
),
                      "core": int(mt.core)})
    lin, anc = un.vector("linear"), un.vector("anchored")
    eq = np.full(un.q, 1 / un.q)
    m0l, m0a = lin.copy(), anc.copy()
    m0l[0] = m0a[0] = 0.0
    t5 = {"gate_linear": agg.hard_gate(lin, w, un.core), "gate_anchored": agg.hard_gate(anc, w, un.core),
          "equal_linear": agg.additive(lin, eq), "equal_anchored": agg.additive(anc, eq),
          "no_source_additive_linear": agg.additive(m0l, w), "no_source_additive_anchored": agg.additive(m0a, w
),
          "no_source_gate": agg.hard_gate(m0l, w, un.core), "no_source_power": agg.power(m0l, w, un.core),
          "no_source_capped": agg.capped(m0l, w, un.core, 0.5)}
    contrib_loss = {f"M{p + 1}": float(w[p] * (anc[p] - lin[p])) for p in range(un.q)}
    R["table5"] = {"rows": rows5, **t5, "contribution_change_anchored_minus_linear": contrib_loss}
    check("UN.S1.A1 gated linear", t5["gate_linear"], 0.9035, 5e-5, "Table 5")
    check("UN.S1.A1 gated anchored", t5["gate_anchored"], 0.8558, 5e-5, "Table 5")
    check("UN.S1.A1 equal weights linear", t5["equal_linear"], 0.8371, 5e-5, "Table 5")
    check("UN.S1.A1 equal weights anchored", t5["equal_anchored"], 0.8361, 5e-5, "Table 5")
    check("UN.S1.A1 additive, M1 = 0, linear", t5["no_source_additive_linear"], 0.6535, 5e-5, "Table 5")
    check("UN.S1.A1 additive, M1 = 0, anchored", t5["no_source_additive_anchored"], 0.6058, 5e-5, "Table 5")
    check("non-compensatory operators, M1 = 0", max(t5["no_source_gate"], t5["no_source_power"],
                                                    t5["no_source_capped"]), 0, 0, "Sec. 7.3")
```

```
    check("difference linear - anchored", t5["gate_linear"] - t5["gate_anchored"], 0.0477, 5e-5, "Sec. 7.2")
    check("loss of contribution M3", -contrib_loss["M3"], 0.0404, 5e-5, "Sec. 7.2")
    check("loss of contribution M4", -contrib_loss["M4"], 0.0200, 5e-5, "Sec. 7.2")
    check("gain of contribution M5", contrib_loss["M5"], 0.0128, 5e-5, "Sec. 7.2")

    # ------------------------------------------------------ Sec. 6 / 7.3: knockout
    ko = {}
    for meth in ("linear", "anchored"):
        m1 = un.vector(meth)
        m1[0] = float(un.metrics[0].table.normalize(1, meth))
        ko[meth] = {"one_source_m1": m1[0], "gate": agg.hard_gate(m1, w, un.core),
                    "power_g1": agg.power(m1, w, un.core, 1.0), "capped_0.5": agg.capped(m1, w, un.core, 0.5),
                    "four_sources_capped_0.5": agg.capped(un.vector(meth), w, un.core, 0.5)}
    ko["C_rest_linear"] = float(w[1:] @ lin[1:])
    ko["compensation_capacity_additive"] = agg.compensation_capacity_additive(w, 0)
    R["knockout"] = ko
    check("C = sum_{q>=2} w_q m_q", ko["C_rest_linear"], 0.6535, 5e-5, "Fig. 7")
    check("C_1(S_w)", ko["compensation_capacity_additive"], 0.75, 1e-12, "Sec. 6.1")
    check("one source, gate, linear", ko["linear"]["gate"], 0.716, 5e-4, "Sec. 7.3 / Table 4")
    check("one source, gate, anchored", ko["anchored"]["gate"], 0.656, 5e-4, "Sec. 7.3")
    check("one source, power γ=1, linear", ko["linear"]["power_g1"], 0.179, 5e-4, "Sec. 6.2 / Table 4")
    check("one source, power γ=1, anchored", ko["anchored"]["power_g1"], 0.131, 5e-4, "Sec. 7.3")
    check("one source, capped ϑ=0.5, linear", ko["linear"]["capped_0.5"], 0.500, 5e-4, "Table 4")
    check("one source, capped ϑ=0.5, anchored", ko["anchored"]["capped_0.5"], 0.400, 5e-4, "Sec. 7.3")
    check("four sources, capped ϑ=0.5, linear", ko["linear"]["four_sources_capped_0.5"], 0.9035, 5e-5, "Sec. 8.
↪ 1")

    # operator properties (Table 4 / Corollary 2) — numerical probes
    K = un.core
    ops = {"additive": lambda m: agg.additive(m, w), "hard_gate": lambda m: agg.hard_gate(m, w, K),
           "power": lambda m: agg.power(m, w, K, 1.0), "capped": lambda m: agg.capped(m, w, K, 0.5)}
    R["operator_properties"] = {
        k: {"idempotent": agg.is_idempotent(f, un.q), "veto": agg.has_veto(f, un.q, 0),
            "jump_at_core_zero": agg.max_jump_at(f, lin, 0)} for k, f in ops.items()}

    # ------------------------------------------------------ Sec. 7.4: sensitivity
    S = {}
    for meth in ("linear", "anchored"):
        _, _, Z = sens.grid(un, 2, 3, (0, 200), (0, 260), meth)
        S[meth] = {"min": float(Z.min()), "max": float(Z.max())}
        S[meth + "_Z"] = Z
    S["max_abs_diff"] = float(np.abs(S["anchored_Z"] - S["linear_Z"]).max())
    S["margin_linear"] = t5["gate_linear"] - 0.9
    S["linear_tau4_threshold"] = sens.threshold(un, 3, 0.9, "linear", 45, 120)
    S["linear_tau3_threshold"] = sens.threshold(un, 2, 0.9, "linear", 25, 90)
    S["anchored_tau3_threshold_at_tau4_45"] = sens.threshold(un, 2, 0.9, "anchored", 1e-6, 24.99)
    S["anchored_tau4_threshold_at_tau3_10"] = sens.threshold(un, 3, 0.9, "anchored", 15.0001, 44.99, {2: 10})
    del S["linear_Z"], S["anchored_Z"]
    R["sensitivity"] = S
    check("grid min, linear", S["linear"]["min"], 0.502, 5e-4, "Sec. 7.4")
    check("grid max, linear", S["linear"]["max"], 0.952, 5e-4, "Sec. 7.4")
    check("grid min, anchored", S["anchored"]["min"], 0.515, 5e-4, "Sec. 7.4")
    check("grid max, anchored", S["anchored"]["max"], 0.965, 5e-4, "Sec. 7.4")
    check("max |anchored - linear|", S["max_abs_diff"], 0.100, 5e-4, "Sec. 7.4")
    check("margin above 0.9 (linear)", S["margin_linear"], 0.0035, 5e-5, "Sec. 7.4")
    check("τ4 threshold, linear", S["linear_tau4_threshold"], 48.9, 0.05, "Sec. 7.4")
    check("τ3 threshold, linear", S["linear_tau3_threshold"], 27.4, 0.05, "Sec. 7.4")
    check("τ3 threshold, anchored (τ4 = 45)", S["anchored_tau3_threshold_at_tau4_45"], 7.3, 0.05, "Sec. 7.4")
    check("τ4 threshold, anchored (τ3 = 10)", S["anchored_tau4_threshold_at_tau3_10"], 37.5, 0.05, "Sec. 7.4")

    # ------------------------------------------------------ Sec. 7.5: weights
    D = {}
    for meth, m in (("linear", lin), ("anchored", anc)):
        for a in (1, 4):
            D[f"{meth}_alpha{a}"] = unc.summary(m, a, n_mc)
    R["dirichlet"] = D
    check("exact mean, linear", D["linear_alpha1"]["exact_mean"], 0.8371, 5e-5, "Sec. 7.5")
    check("exact sd, linear (α=1)", D["linear_alpha1"]["exact_sd"], 0.0716, 5e-5, "Sec. 7.5")
    check("exact mean, anchored", D["anchored_alpha1"]["exact_mean"], 0.8361, 5e-5, "Sec. 7.5")
    check("exact sd, anchored (α=1)", D["anchored_alpha1"]["exact_sd"], 0.0452, 5e-5, "Sec. 7.5")
    for k in ("linear_alpha1", "anchored_alpha1"):
        check(f"MC vs exact mean ({k})", D[k]["mc_mean"], D[k]["exact_mean"], 2e-4 if not args.fast else 1e-3,
↪ "Sec. 7.5")
        check(f"MC vs exact sd ({k})", D[k]["mc_sd"], D[k]["exact_sd"], 2e-4 if not args.fast else 1e-3, "Sec.
↪ 7.5")
    check("P(high), linear, α=1", D["linear_alpha1"]["p_high"], 0.185, mc_tol, "Sec. 7.5")
    check("P(high), anchored, α=1", D["anchored_alpha1"]["p_high"], 0.095, mc_tol, "Sec. 7.5")
    check("P(high), linear, α=4", D["linear_alpha4"]["p_high"], 0.029, mc_tol, "Sec. 7.5")
    check("P(high), anchored, α=4", D["anchored_alpha4"]["p_high"], 0.005, mc_tol, "Sec. 7.5")
    check("P(<0.7), linear, α=1", D["linear_alpha1"]["p_below_0.7"], 0.050, mc_tol, "Sec. 7.5")
    check("P(<0.7), anchored, α=1", D["anchored_alpha1"]["p_below_0.7"], 0.0, 0, "Prop. 3")

    # ------------------------------------------------------ Table 6 / Sec. 7.6
    sc = {m: crtm.evaluate_scenario(actions, method=m) for m in ("linear", "anchored")}
    pub6 = {"UN.S1.A1": (0.9035, 0.8558), "UN.S2.A1": (0.6437, 0.6702), "CN.S1.A1": (0.5905, 0.5885),
            "CN.S2.A1": (0.6173, 0.5989), "RE.S1.A2": (0.7639, 0.7192), "TR.S1.A1": (0.6967, 0.6817),
```

```
            "TR.S2.A2": (0.8036, 0.7533)}
    for b, (pl, pa) in pub6.items():
        check(f"{b} linear", sc["linear"]["actions"][b].score, pl, 5e-5, "Table 6")
        check(f"{b} anchored", sc["anchored"]["actions"][b].score, pa, 5e-5, "Table 6")
    c2 = {m: sc[m]["actions"]["CN.S2.A1"] for m in sc}
    check("CN.S2.A1 κ", c2["anchored"].kappa, 0.9, 1e-12, "Table 6")
    check("CN.S2.A1 enclosure low, linear", c2["linear"].lower, 0.556, 5e-4, "Table 6")
    check("CN.S2.A1 enclosure high, linear", c2["linear"].upper, 0.656, 5e-4, "Table 6")
    check("CN.S2.A1 enclosure low, anchored", c2["anchored"].lower, 0.539, 5e-4, "Table 6")
    check("CN.S2.A1 enclosure high, anchored", c2["anchored"].upper, 0.639, 5e-4, "Table 6")
    pub_goal = {"linear": {"ANT": (0.774, 0.644), "WITH": (0.604, 0.591), "REC": (0.764, 0.764), "AD": (0.750,
 0.697)},
                "anchored": {"ANT": (0.763, 0.670), "WITH": (0.594, 0.588), "REC": (0.719, 0.719), "AD": (0.718
, 0.682)}}
    for m, gs in pub_goal.items():
        for g, (mean, mn) in gs.items():
            check(f"goal {g} mean, {m}", sc[m]["profile"][g]["mean"], mean, 5e-4, "Table 6")
            check(f"goal {g} min, {m}", sc[m]["profile"][g]["min"], mn, 5e-4, "Table 6")
    check("linear profile dominates anchored", int(crtm.dominates(sc["linear"]["profile"], sc["anchored"]["prof
ile"])),
          1, 0, "Sec. 7.6")
    for m in sc:
        capped_sc = crtm.evaluate_scenario(actions, method=m, operator="capped", theta=0.5)["actions"]
        binds = sum(not math.isclose(capped_sc[b].score, sc[m]["actions"][b].score) for b in actions)
        check(f"actions where cap ϑ=0.5 binds ({m})", binds, 0, 0, "Sec. 7.6")
    R["scenario"] = {m: {"actions": {b: r.as_dict() for b, r in sc[m]["actions"].items()},
                         "profile": sc[m]["profile"], "N_theta": sc[m]["N_theta"]} for m in sc}
    check("N_theta (metrics in scenario)", sc["anchored"]["N_theta"], 39, 0, "Sec. 7.1")

    # ---------------------------------------------------------------- write files
    with open(out / "tables" / "TableA1_phishing_metrics.csv", "w", newline="", encoding="utf-8") as fh:
        wr = csv.writer(fh)
        wr.writerow(["action", "p", "code", "metric", "unit", "printed_classes", "xi", "sigma", "value",
                     "tab_level", "n_ch", "lvl_lin", "nu", "lvl_anc", "consistent_linear", "w", "core"])
        for b, a in actions.items():
            for p, mt in enumerate(a.metrics):
                nl, na = mt.normalized("linear"), mt.normalized("anchored")
                wr.writerow([b, p + 1, mt.code, mt.name, mt.unit, mt.printed_classes,
                             " ".join(f"{v:g}" for v in mt.table.xi), mt.sigma,
                             "" if not mt.observed else f"{mt.value:g}",
                             crtm.LEVEL_ABBR[mt.tabulated_level() - 1] if mt.observed else "n.r.",
                             "" if math.isnan(nl) else f"{nl:.4f}", crtm.level_abbr(nl),
                             "" if math.isnan(na) else f"{na:.4f}", crtm.level_abbr(na),
                             "" if not mt.observed else int(crtm.level(nl) == mt.tabulated_level()),
                             f"{a.w()[p]:.4f}", int(mt.core)])
    with open(out / "tables" / "Table5_UN.S1.A1.csv", "w", newline="", encoding="utf-8") as fh:
        wr = csv.DictWriter(fh, fieldnames=list(rows5[0]))
        wr.writeheader()
        wr.writerows(rows5)
    with open(out / "tables" / "Table6_scenario.csv", "w", newline="", encoding="utf-8") as fh:
        wr = csv.writer(fh)
        wr.writerow(["action", "goal", "q", "core", "kappa", "linear", "lvl_lin", "lin_low", "lin_high",
                     "anchored", "lvl_anc", "anc_low", "anc_high", "inconsistent_linear"])
        for b, a in actions.items():
            rl, ra = sc["linear"]["actions"][b], sc["anchored"]["actions"][b]
            wr.writerow([b, a.goal, a.q, "/".join(a.metrics[k].code for k in a.core) or "—",
                         f"{ra.kappa:.2f}", f"{rl.score:.4f}", crtm.level_abbr(rl.score), f"{rl.lower:.4f}",
                         f"{rl.upper:.4f}", f"{ra.score:.4f}", crtm.level_abbr(ra.score), f"{ra.lower:.4f}",
                         f"{ra.upper:.4f}", len(rl.inconsistent)])
        for g in sc["anchored"]["profile"]:
            pl, pa = sc["linear"]["profile"][g], sc["anchored"]["profile"][g]
            wr.writerow([f"goal {g} (mean/min)", g, "", "", "", f"{pl['mean']:.3f}/{pl['min']:.3f}", "", "", ""
,
                         f"{pa['mean']:.3f}/{pa['min']:.3f}", "", "", "", ""])

    R["checks"] = CHECKS
    with open(out / "results.json", "w", encoding="utf-8") as fh:
        json.dump(jsonable(R), fh, indent=2, ensure_ascii=False)
    with open(out / "checks.csv", "w", newline="", encoding="utf-8") as fh:
        wr = csv.DictWriter(fh, fieldnames=list(CHECKS[0]))
        wr.writeheader()
        wr.writerows(CHECKS)

    if not args.no_figures:
        import matplotlib
        if "ipykernel" not in sys.modules:
            matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        from crtm import figures as F
        fdir = out / "figures"
        fdir.mkdir(exist_ok=True)
        for name, (arg, fn) in F.ALL.items():
            obj = scen if arg == "scen" else actions if arg == "actions" else actions[arg]
            fig = fn(obj)
            for ext in ("png", "svg", "pdf"):
                fig.savefig(fdir / f"{name}.{ext}", bbox_inches="tight", dpi=300)
```

```
            plt.close(fig)
            print(f"  figure  {name}")

    n_bad = sum(not c["ok"] for c in CHECKS)
    width = max(len(c["quantity"]) for c in CHECKS)
    for c in CHECKS:
        flag = "OK " if c["ok"] else "FAIL"
        print(f"  [{flag}] {c['quantity']:<{width}}  computed {c['computed']:>10.4f}   "
              f"published {c['published']:>10.4f}   ({c['where']})")
    print(f"\n{len(CHECKS) - n_bad}/{len(CHECKS)} published values reproduced; results in {out}")
    return 1 if n_bad else 0


if __name__ == "__main__":
    sys.exit(main())
```

tests/conftest.py

SHA-256 c50c8305f585c9551191788e47ab0cbebac1b54229902d72a7bd11ce8f5ae4b5

```
1 import sys
2 from pathlib import Path
3
4 sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
```

tests/test_aggregation.py

SHA-256 51a332a0e142ab92c235a4da1b58f3ffa12fa945dd8a2e72793c1243110b1f7b

```python
"""Propositions 4-7, Theorem 3 and Corollary 2."""
import numpy as np
import pytest

from crtm import aggregation as agg
from crtm import uncertainty as unc

W = np.array([0.25, 0.15, 0.25, 0.20, 0.05, 0.10])
K = [0]


def ops():
    return {"gate": lambda m: agg.hard_gate(m, W, K),
            "power": lambda m: agg.power(m, W, K, 1.0),
            "capped": lambda m: agg.capped(m, W, K, 0.5)}


def test_idempotency_table4():
    o = ops()
    assert agg.is_idempotent(lambda m: agg.additive(m, W), 6)
    assert agg.is_idempotent(o["gate"], 6)
    assert not agg.is_idempotent(o["power"], 6)
    assert agg.is_idempotent(o["capped"], 6)


def test_veto_table4():
    o = ops()
    assert not agg.has_veto(lambda m: agg.additive(m, W), 6, 0)
    for f in o.values():
        assert agg.has_veto(f, 6, 0)


def test_continuity_corollary2():
    m = np.array([0.0, 1, 0.9, 0.87, 0.44, 0.8])
    o = ops()
    assert agg.max_jump_at(o["gate"], m, 0) > 0.5            # Prop. 4(c): jump = S_w(m)
    assert agg.max_jump_at(o["power"], m, 0) < 1e-6
    assert agg.max_jump_at(o["capped"], m, 0) < 1e-6


def test_theorem3_lipschitz_and_limits():
    rng = np.random.default_rng(3)
    for th in (0.25, 0.5, 1.0):
        L = max(1, 1 / th)
        for _ in range(500):
            a, b = rng.random(6), rng.random(6)
            assert abs(agg.capped(a, W, K, th) - agg.capped(b, W, K, th)) <= L * np.abs(a - b).max() + 1e-12
    m = rng.random(6)
    # (c) capped <= gate; (d) convergence to the gate for small theta
    assert agg.capped(m, W, K, 0.5) <= agg.hard_gate(m, W, K) + 1e-12
    assert agg.capped(m, W, K, min(m[0], 1.0)) == pytest.approx(agg.hard_gate(m, W, K))


def test_prop5d_nonuniform_convergence():
    for g in (0.1, 0.5, 1, 2):
        m = np.array([1e-300, 1, 1, 1, 1, 1])
        assert agg.hard_gate(m, W, K) - agg.power(m, W, K, g) >= 1 - W[0] - 1e-3


def test_prop6_enclosure_attained():
    rng = np.random.default_rng(4)
    w = np.full(10, 0.1)
    m = rng.random(10)
    m[8] = np.nan
    enc = agg.missing_enclosure(m, w, [2])
    for fill in (0.0, 1.0):
        full = np.where(np.isnan(m), fill, m)
        s = agg.hard_gate(full, w, [2])
        assert s == pytest.approx(enc["lower"] if fill == 0 else enc["upper"])
    for _ in range(200):
        full = np.where(np.isnan(m), rng.random(), m)
        s = agg.hard_gate(full, w, [2])
        assert enc["lower"] - 1e-12 <= s <= enc["upper"] + 1e-12
        assert abs(s - enc["mu_O"]) <= enc["max_deviation"] + 1e-12


def test_missing_core_not_computed():
    m = np.array([np.nan, 0.5, 0.5])
    assert agg.missing_enclosure(m, np.ones(3) / 3, [0])["status"].startswith("not computed")


def test_prop7_dirichlet_moments():
    m = np.array([1, 1, 0.9118, 0.8667, 0.4444, 0.8])
    for a in (0.5, 1, 4):
        mean, sd = unc.dirichlet_moments(m, a)
```

```
        s = unc.sample_scores(m, a, n=200_000, seed=7)
        assert s.mean() == pytest.approx(mean, abs=1e-3)
        assert s.std() == pytest.approx(sd, abs=1e-3)
```

tests/test_hierarchy.py

SHA-256 ff5b820a52f5a9a357136805a9b96b114986b57de313820f73054ee5f96ac3e9

```
"""Proposition 1 (identifiers) and Proposition 2 (lattice of projections)."""
import itertools
import random

import pytest

import crtm
from crtm.hierarchy import Projection, check_code, iota, iota_inv, jaccard_index


def test_iota_roundtrip_and_injective():
    ids = {}
    for i in crtm.TASKS:
        for j in range(1, 12):
            for k in range(1, 12):
                s = iota(i, j, k)
                assert iota_inv(s) == (i, j, k)
                ids[(i, j, k)] = s
    assert len(set(ids.values())) == len(ids)
    assert iota(6, 1, 1) == "UN.S1.A1"


def test_homoglyph_detection():
    assert check_code("CN") == []
    assert check_code("CN")          # Cyrillic Es looks like Latin C
    with pytest.raises(ValueError):
        iota_inv("CN.S1.A1")


def test_technique_codes_injective():
    assert crtm.hierarchy.is_injective(crtm.TECHNIQUES)
    assert len(crtm.TECHNIQUES) == 14


@pytest.fixture(scope="module")
def scen():
    return crtm.load_scenarios()


def test_projections_admissible(scen):
    for p in scen["projections"].values():
        assert p.is_admissible(scen["catalog"])
    bad = Projection({"UN.S3.A1": {"CA"}})
    assert not bad.is_admissible(scen["catalog"])     # example of Section 3.3


def test_section_3_5_numbers(scen):
    P, Q = scen["projections"]["DDoS"], scen["projections"]["phishing"]
    assert sorted((P & Q).actions) == ["CN.S1.A1", "TR.S1.A1", "TR.S2.A2", "UN.S1.A1"]
    assert len((P | Q).actions) == 12
    assert jaccard_index(P.actions, Q.actions) == pytest.approx(1 / 3)
    assert (len(P.downset()), len(Q.downset())) == (26, 23)
    assert jaccard_index(P.downset(), Q.downset()) == pytest.approx(13 / 36)


def _random_projection(cat, rng):
    return Projection({b: {t for t in ts if rng.random() < 0.6}
                       for b, ts in cat.items() if rng.random() < 0.5})


def test_proposition2_distributive_lattice(scen):
    cat = scen["catalog"]
    rng = random.Random(5)
    Ps = [_random_projection(cat, rng) for _ in range(12)]
    for A, B, C in itertools.product(Ps[:6], repeat=3):
        J, M = A | B, A & B
        assert J.is_admissible(cat) and M.is_admissible(cat)
        assert A <= J and B <= J and M <= A and M <= B
        assert A & (B | C) == (A & B) | (A & C)
        assert A | (B & C) == (A | B) & (A | C)
        # order embedding into down-sets
        assert (A <= B) == (A.downset() <= B.downset())
```

tests/test_normalization.py

SHA-256 95a42f184f72f638641a5cae8e01845af250c54e00fdab4de51327844013cd54

```python
"""Theorems 1-2, Corollary 1 and Proposition 3."""
import numpy as np
import pytest

import crtm
from crtm.normalization import ClassTable, ClassTableError, grid_check


@pytest.fixture(scope="module")
def actions():
    return crtm.load_metrics()


def test_level_map_bands():
    assert crtm.level(0.0) == 1 and crtm.level(0.1999) == 1
    assert crtm.level(0.2) == 2 and crtm.level(0.7) == 4
    assert crtm.level(0.9) == 5 and crtm.level(1.0) == 5


def test_condition_8_rejected():
    with pytest.raises(ClassTableError):
        ClassTable((0, 1, 1, 2, 3, 4))


def test_paper_encoding_relevance_time():
    t = ClassTable((180, 179, 90, 30, 10, 0))
    assert t.sigma == -1
    assert t.tabulated_level(25) == 4          # 11-30 min: above average
    assert t.linear(25) == pytest.approx(0.9118, abs=1e-4)
    assert crtm.level(t.linear(25)) == 5      # ... but the linear map says "high"
    assert t.anchored(25) == pytest.approx(0.75)


def test_theorem2a_anchored_consistent_everywhere(actions):
    for a in actions.values():
        for mt in a.metrics:
            assert grid_check(mt.table, "anchored")["violations"] == 0


def test_corollary1_linear_never_consistent(actions):
    for a in actions.values():
        for mt in a.metrics:
            t = mt.table
            c = t.corollary1_interval()
            assert c["length"] > 0
            u = np.linspace(*c["u_interval"], 50)[1:-1]   # interior (avoid rounding at 0.9)
            x = t.sigma * u
            assert np.all(crtm.level(t.linear(x)) == 5)
            assert np.all(t.tabulated_level(x) == 4)


def test_theorem1_even_spacing_gives_consistent_affine_map():
    # anchors spaced proportionally to y: slopes 100 per unit score
    t = ClassTable((0, 20, 40, 70, 90, 100))
    ok, a, b = t.affine_consistent()
    assert ok and a == pytest.approx(0.01) and b == pytest.approx(0.0)
    xs = np.linspace(-10, 110, 5001)
    assert np.all(crtm.level(t.anchored(xs)) == t.tabulated_level(xs))
    # Theorem 2(c): nu coincides with the affine map
    assert np.allclose(t.anchored(xs), np.clip(a * xs + b, 0, 1))


def test_theorem2d_band_width_bound(actions):
    rng = np.random.default_rng(1)
    for a in actions.values():
        for mt in a.metrics:
            t = mt.table
            xs = rng.uniform(min(t.xi), max(t.xi), 200)
            r = t.tabulated_level(xs)
            # any consistent normalization: a random point inside the same band
            lo, hi = crtm.Y[r - 1], crtm.Y[r]
            other = lo + rng.random(xs.size) * (hi - lo) * 0.999
            assert np.all(np.abs(other - t.anchored(xs)) <= (hi - lo) + 1e-12)
            assert np.all(np.abs(other - t.anchored(xs)) <= 0.3 + 1e-12)


def test_proposition3_level_bounds():
    rng = np.random.default_rng(2)
    for _ in range(2000):
        q = rng.integers(2, 8)
        m = rng.random(q)
        w = rng.dirichlet(np.ones(q))
        lv = crtm.level(m)
        assert lv.min() <= crtm.level(w @ m) <= lv.max()

```

```
86
87 def test_degenerate_critical_anchor_is_consistent():
88     t = ClassTable((5, 5, 10, 20, 30, 40))
89     xs = np.concatenate([np.linspace(0, 45, 9001), t.anchors_with_neighbours()])
90     assert np.all(crtm.level(t.anchored(xs)) == t.tabulated_level(xs))
```

tests/test_reproduction.py

SHA-256 d5277bea807079b4d3d93c7d02222b562ef421b7809af69b6a869a89b31945de

```
"""End-to-end: all published values of Section 7 are reproduced."""
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent


def test_reproduce_script(tmp_path):
    r = subprocess.run([sys.executable, str(ROOT / "scripts" / "reproduce.py"), "--out", str(tmp_path),
                        "--no-figures", "--fast"], capture_output=True, text=True)
    assert r.returncode == 0, r.stdout[-3000:] + r.stderr[-3000:]
    assert (tmp_path / "results.json").exists()
```

tools/build_listing.py

SHA-256 5b776746f47660ccff92a653dbd03080e2d774f5efee630344a6fb5407056523

```
#!/usr/bin/env python3
"""Build the program-code listing docs/LISTING.pdf and docs/LISTING.md.

Each source file is printed with line numbers, its size and SHA-256 digest,
so the listing can be matched against the archived software version.
"""
from __future__ import annotations

import datetime as dt
import hashlib
from pathlib import Path

from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.pdfgen import canvas

ROOT = Path(__file__).resolve().parent.parent
import sys  # noqa: E402

sys.path.insert(0, str(ROOT))
from crtm import __version__  # noqa: E402

FILES = [*sorted(ROOT.glob("crtm/*.py")), ROOT / "scripts" / "reproduce.py",
         *sorted(ROOT.glob("tests/*.py")), *sorted(ROOT.glob("tools/*.py"))]
TITLE = "CRTM — Cyber-Resilience Tuple Model: program code listing"

FONT_DIRS = [Path("/usr/share/fonts/truetype/dejavu"), Path("/usr/share/fonts/dejavu"),
             Path("/Library/Fonts"), Path("C:/Windows/Fonts")]


def _register_fonts() -> tuple[str, str]:
    for d in FONT_DIRS:
        mono, sans = d / "DejaVuSansMono.ttf", d / "DejaVuSans.ttf"
        if mono.exists() and sans.exists():
            pdfmetrics.registerFont(TTFont("Mono", str(mono)))
            pdfmetrics.registerFont(TTFont("Sans", str(sans)))
            bold = d / "DejaVuSans-Bold.ttf"
            pdfmetrics.registerFont(TTFont("SansB", str(bold if bold.exists() else sans)))
            return "Mono", "Sans"
    raise SystemExit("DejaVu fonts not found (needed for Greek and math symbols)")


def sha(p: Path) -> str:
    return hashlib.sha256(p.read_bytes()).hexdigest()


def build_pdf(out: Path):
    mono, sans = _register_fonts()
    W, H = A4
    c = canvas.Canvas(str(out), pagesize=A4)
    c.setTitle(TITLE)
    c.setAuthor("[Author One], [Author Two], [Author Three]")
    left, top, bottom = 15 * mm, H - 18 * mm, 15 * mm
    fs, lh = 7.2, 8.6
    max_chars = int((W - left - 15 * mm - 9 * mm) / pdfmetrics.stringWidth("M", mono, fs))
    page = [1]

    def footer():
        c.setFont(sans, 7)
        c.setFillGray(0.4)
        c.drawString(left, 9 * mm, f"crtm {__version__} — program code listing")
        c.drawRightString(W - 15 * mm, 9 * mm, f"page {page[0]}")
        c.setFillGray(0)

    def new_page():
        footer()
        c.showPage()
        page[0] += 1

    # title page
    c.setFont("SansB", 15)
    c.drawString(left, top - 10 * mm, "CRTM — Cyber-Resilience Tuple Model")
    c.setFont(sans, 11)
    c.drawString(left, top - 18 * mm, f"Program code listing, software version {__version__}")
    c.setFont(sans, 8.5)
    lines = ["Digital supplement (Code S1) to the article “A Set-Theoretic Tuple Model with Level-Consistent",
             "Normalization and Non-Compensatory Aggregation for Assessing the Cyber Resilience of Critical",
             "Infrastructure”, Information (MDPI), 2026. License: MIT (code), CC BY 4.0 (data, figures).",
             f"Generated {dt.date.today().isoformat()} by tools/build_listing.py."]
    y = top - 28 * mm
    for s in lines:
        c.drawString(left, y, s)
        y -= 11
```

```
    y -= 10
    c.setFont("SansB", 9)
    c.drawString(left, y, "File")
    c.drawString(left + 70 * mm, y, "Lines")
    c.drawString(left + 85 * mm, y, "SHA-256")
    y -= 12
    c.setFont(mono, 7)
    total = 0
    for p in FILES:
        n = len(p.read_text(encoding="utf-8").splitlines())
        total += n
        c.drawString(left, y, str(p.relative_to(ROOT)))
        c.drawRightString(left + 80 * mm, y, str(n))
        c.drawString(left + 85 * mm, y, sha(p))
        y -= 10
    c.setFont("SansB", 8)
    c.drawString(left, y - 4, f"{len(FILES)} files, {total} lines")
    new_page()

    for p in FILES:
        rel = str(p.relative_to(ROOT))
        y = top
        c.setFont("SansB", 10)
        c.drawString(left, y, rel)
        c.setFont(sans, 6.5)
        c.setFillGray(0.4)
        c.drawString(left, y - 9, f"SHA-256 {sha(p)}")
        c.setFillGray(0)
        y -= 22
        for i, line in enumerate(p.read_text(encoding="utf-8").splitlines(), 1):
            chunks = [line[k:k + max_chars] for k in range(0, max(len(line), 1), max_chars)] or [""]
            for j, ch in enumerate(chunks):
                if y < bottom:
                    new_page()
                    y = top
                    c.setFont(sans, 7)
                    c.setFillGray(0.4)
                    c.drawString(left, y, f"{rel} (continued)")
                    c.setFillGray(0)
                    y -= 14
                c.setFont(mono, fs)
                c.setFillGray(0.55)
                c.drawRightString(left + 7 * mm, y, str(i) if j == 0 else "↪")
                c.setFillGray(0)
                c.drawString(left + 9 * mm, y, ch)
                y -= lh
        new_page()
    c.save()
    return page[0] - 1


def build_md(out: Path):
    parts = [f"# {TITLE}\n", f"Software version {__version__}. Generated by `tools/build_listing.py`.\n",
             "| File | Lines | SHA-256 |", "|---|---:|---|"]
    for p in FILES:
        parts.append(f"| `{p.relative_to(ROOT)}` | {len(p.read_text(encoding='utf-8').splitlines())} | `{sha(p)
}` |")
    for p in FILES:
        parts += ["", f"## `{p.relative_to(ROOT)}`", "", "```python", p.read_text(encoding="utf-8").rstrip(), "
```"]
    out.write_text("\n".join(parts) + "\n", encoding="utf-8")

if __name__ == "__main__":
    (ROOT / "docs").mkdir(exist_ok=True)
    n = build_pdf(ROOT / "docs" / "LISTING.pdf")
    build_md(ROOT / "docs" / "LISTING.md")
    print(f"wrote docs/LISTING.pdf ({n} pages) and docs/LISTING.md")
```
```

tools/build_notebook.py

SHA-256 3612ba1cd3b4c3f9a8df1becee447ca9c3d9920072337cd343de983d325afbd4

```
#!/usr/bin/env python3
"""Build notebooks/CRTM_Colab.ipynb from the package sources.

The setup cell embeds the package, the data files and the reproduction
script, so the notebook runs stand-alone in Google Colab (no upload, no
network access to the repository needed). When the notebook is opened
inside the unpacked supplement, the local sources are used instead.
"""
from __future__ import annotations

import base64
import json
import zlib
from pathlib import Path

import nbformat as nbf

ROOT = Path(__file__).resolve().parent.parent
EMBED = sorted([*ROOT.glob("crtm/*.py"), *ROOT.glob("data/*"), ROOT / "scripts" / "reproduce.py"])


def md(s):
    return nbf.v4.new_markdown_cell(s.strip())


def code(s):
    return nbf.v4.new_code_cell(s.strip())


def setup_cell() -> str:
    files = {str(p.relative_to(ROOT)): p.read_text(encoding="utf-8") for p in EMBED}
    payload = base64.b64encode(zlib.compress(json.dumps(files, ensure_ascii=False).encode("utf-8"), 9)).decode(
)
    payload = "\n".join(payload[i:i + 100] for i in range(0, len(payload), 100))
    return f'''
# @title ⚙ Setup: locate or unpack the `crtm` package (run first)
import sys, json, pathlib, importlib, base64, zlib

def _find_local():
    for c in [pathlib.Path.cwd(), *pathlib.Path.cwd().parents][:3]:
        if (c / "crtm" / "__init__.py").exists() and (c / "data" / "phishing_metrics.csv").exists():
            return c
    return None

ROOT = _find_local()
if ROOT is None:
    # stand-alone mode (e.g. Google Colab): write the embedded sources
    ROOT = pathlib.Path("crtm-supplement").resolve()
    _FILES = json.loads(zlib.decompress(base64.b64decode("""
{payload}
""")).decode("utf-8"))
    for rel, text in _FILES.items():
        p = ROOT / rel
        p.parent.mkdir(parents=True, exist_ok=True)
        p.write_text(text, encoding="utf-8")
    print(f"unpacked {{len(_FILES)}} embedded files into {{ROOT}}")
sys.path.insert(0, str(ROOT))
import crtm
importlib.reload(crtm)
print("crtm", crtm.__version__, "from", ROOT)
'''


CELLS = [
    md("""
# Cyber-Resilience Tuple Model (CRTM) — interactive digital supplement

Companion notebook for the article

> **A Set-Theoretic Tuple Model with Level-Consistent Normalization and Non-Compensatory
> Aggregation for Assessing the Cyber Resilience of Critical Infrastructure.**
> *Information* (MDPI), 2026. DOI: *to be assigned*. Software DOI (Zenodo): *to be assigned*.

The notebook re-computes every number of the illustrative application (phishing, ATT&CK T1566,
at the bank processing center BNK_PAY), redraws the figures, and lets you change the inputs
interactively. It runs unchanged in **Google Colab** (*Runtime → Run all*) or locally in Jupyter.

| Section | Article | What you can do |
|---|---|---|
| 1 | §3 | identifiers, admissible projections, lattice operations ⊔ / ⊓, Jaccard distance |
| 2 | §4–5 | class tables, Theorem 1, Corollary 1, anchored normalization, consistency check |
| 3 | §6 | knockout operators (hard gate, power, capped) — sliders for γ, θ and the core metric |
| 4 | §7.2 | worked example UN.S1.A1 (Table 5) |
| 5 | §7.4 | sensitivity to response times — sliders for $\tau_3$, $\tau_4$ |
| 6 | §7.5 | random weights (Dirichlet), closed-form moments vs Monte Carlo |
```

```
| 7 | §7.6 | scenario profile, missing-metric enclosure (Table 6) |
| 8 | — | full reproduction check (107 published values) and export |
| 9 | — | sandbox: score your own metric |

*The numerical values are the illustrative example values of the source study, not field measurements.*
"""),
    code(setup_cell()),
    code("""
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from IPython.display import display, Markdown
from crtm import aggregation as agg, sensitivity as sens, uncertainty as unc, figures as F
from crtm.hierarchy import Projection, iota, iota_inv, check_code, jaccard_index
pd.set_option("display.precision", 4)
actions = crtm.load_metrics()
scen = crtm.load_scenarios()
UN = actions["UN.S1.A1"]
try:
    import ipywidgets as W
    HAVE_WIDGETS = True
except ImportError:
    HAVE_WIDGETS = False
print("actions:", list(actions), "| metrics:", sum(a.q for a in actions.values()),
      "| widgets:", HAVE_WIDGETS)
"""),
    md("""
## 1. Set-theoretic data model (Section 3)

**Proposition 1.** The identifier map ι(i, j, k) = c_i.Sj.Ak is injective and is inverted by parsing.
The check below also illustrates the *mixed-script* defect found in the source tabulation: a Cyrillic
«С» that looks like a Latin «C».
"""),
    code("""
print(iota(6, 1, 1), "->", iota_inv("UN.S1.A1"))
print("Latin 'CN'   :", check_code("CN") or "valid")
print("Cyrillic 'CN':", check_code("\\u0421N"))
"""),
    md("""
**Definition 3 / Proposition 2.** A projection is *admissible* if it uses only catalog techniques of each
action. Admissible projections form a finite distributive lattice: join ⊔ = compound threat,
meet ⊓ = common actions.
"""),
    code("""
P, Q = scen["projections"]["DDoS"], scen["projections"]["phishing"]
J, M = P | Q, P & Q
print("admissible:", P.is_admissible(scen["catalog"]), Q.is_admissible(scen["catalog"]))
print("meet  P⊓Q :", sorted(M.actions))
print("|join| =", len(J.actions), " Jaccard(actions) =", round(jaccard_index(P.actions, Q.actions), 4))
print("|E(P)|, |E(Q)| =", len(P.downset()), len(Q.downset()),
      " Jaccard(down-sets) =", round(jaccard_index(P.downset(), Q.downset()), 4))
print("inadmissible example:", Projection({"UN.S3.A1": {"CA"}}).violations(scen["catalog"]))
F.fig_projections(scen); plt.show()
"""),
    md("""
## 2. Metric model and level-consistent normalization (Sections 4–5)

All 39 metrics of the phishing scenario with their class tables ξ (Appendix A).
`tab` is the level printed in the class table; `lin`/`anc` are the levels of the two normalizations.
"""),
    code("""
rows = []
for b, a in actions.items():
    for p, mt in enumerate(a.metrics):
        nl, na = mt.normalized("linear"), mt.normalized("anchored")
        rows.append(dict(action=b, p=p+1, code=mt.code + ("*" if mt.core else ""), metric=mt.name,
                         unit=mt.unit, xi=mt.table.xi, value=mt.value,
                         tab=crtm.LEVEL_ABBR[mt.tabulated_level()-1] if mt.observed else "n.r.",
                         n_ch=nl, lin=crtm.level_abbr(nl), nu=na, anc=crtm.level_abbr(na)))
tA1 = pd.DataFrame(rows)
tA1["linear consistent"] = (tA1["lin"] == tA1["tab"]) | (tA1["tab"] == "n.r.")
display(tA1)
print("inconsistent linear scores:", (~tA1["linear consistent"]).sum(), "of", (tA1["tab"] != "n.r.").sum())
"""),
    md("""
**Theorem 1 / Corollary 1.** A clipped affine map can match a five-level class table only if the
class boundaries are evenly spaced relative to y = (0, .2, .4, .7, .9, 1). The two-anchor rule
n_{c,h} sends η₄ to 1 ≠ 0.9 and is therefore *never* consistent: the interval below is always nonempty.
"""),
    code("""
t3 = UN.metrics[2].table
print("Theorem 1 condition (11) holds:", t3.affine_consistent()[0])
print("Corollary 1 interval for M3:", t3.corollary1_interval())
F.fig_normalizations(UN); plt.show()
F.fig_consistency(actions); plt.show()
"""),
    md("""
## 3. Non-compensatory aggregation (Section 6)
```

```
171
172 Move the sliders: number of threat-intelligence sources (the knockout metric M1 of UN.S1.A1),
173 power γ and cap ϑ. Watch the **jump** of the hard gate between 0 and 1 source, the loss of
174 **idempotency** of power discounting, and the capped operator that keeps all three properties
175 (Corollary 2).
176 """),
177     code("""
178 def knockout_demo(sources=1, method="linear", gamma=1.0, theta=0.5):
179     m = UN.vector(method); w = UN.w()
180     m[0] = float(UN.metrics[0].table.normalize(sources, method))
181     vals = {"additive S_w": agg.additive(m, w), "hard gate μ^Φ": agg.hard_gate(m, w, UN.core),
182             f"power γ={gamma}": agg.power(m, w, UN.core, gamma),
183             f"capped ϑ={theta}": agg.capped(m, w, UN.core, theta)}
184     df = pd.DataFrame({"score": vals}).assign(level=lambda d: [crtm.level_name(v) for v in d.score])
185     display(Markdown(f"**{sources} source(s)** → normalized m₁ = {m[0]:.3f} ({method})"))
186     display(df)
187
188 if HAVE_WIDGETS:
189     W.interact(knockout_demo, sources=W.IntSlider(1, 0, 4), method=["linear", "anchored"],
190                gamma=W.FloatSlider(value=1.0, min=0.1, max=3, step=0.1),
191                theta=W.FloatSlider(value=0.5, min=0.05, max=1, step=0.05))
192 else:
193     for s in (0, 1, 4): knockout_demo(s)
194 """),
195     code("""
196 w, K = UN.w(), UN.core
197 props = {name: {"idempotent": agg.is_idempotent(f, 6), "veto at M1": agg.has_veto(f, 6, 0),
198                 "jump at m1=0": round(agg.max_jump_at(f, UN.vector("linear"), 0), 4)}
199          for name, f in {"additive": lambda m: agg.additive(m, w), "hard gate": lambda m: agg.hard_gate(m, w, K
    ↪ ),
200                          "power γ=1": lambda m: agg.power(m, w, K, 1), "capped ϑ=0.5": lambda m: agg.capped(m,
    ↪ w, K, .5)}.items()}
201 display(pd.DataFrame(props).T)   # cf. Table 4
202 F.fig_operators(UN); plt.show()
203 """),
204     md("## 4. Worked example: threat-intelligence action UN.S1.A1 (Table 5)"),
205     code("""
206 w = UN.w(); lin, anc = UN.vector("linear"), UN.vector("anchored")
207 t5 = pd.DataFrame({"metric": [m.name for m in UN.metrics], "value": [m.value for m in UN.metrics],
208                    "n_ch": lin, "lvl": [crtm.level_abbr(v) for v in lin], "ν_ξ": anc,
209                    "lvl ": [crtm.level_abbr(v) for v in anc], "w": w, "w·n": w*lin, "w·ν": w*anc})
210 display(t5)
211 for name, a, b in (("gated, specified w", agg.hard_gate(lin, w, K), agg.hard_gate(anc, w, K)),
212                    ("equal weights", agg.additive(lin, np.ones(6)/6), agg.additive(anc, np.ones(6)/6))):
213     print(f"{name:20s} linear {a:.4f} ({crtm.level_abbr(a)})   anchored {b:.4f} ({crtm.level_abbr(b)})")
214 F.fig_contributions(UN); plt.show()
215 """),
216     md("""
217 ## 5. Sensitivity to response times (Section 7.4)
218
219 The linear verdict *high* holds only by a margin of 0.0035. Drag τ₃ (relevance) and τ₄
220 (dissemination) to see where each normalization changes level.
221 """),
222     code("""
223 def times_demo(tau3=25.0, tau4=45.0):
224     for meth in ("linear", "anchored"):
225         s = sens.score_with(UN, {2: tau3, 3: tau4}, meth)
226         print(f"{meth:9s} score {s:.4f}  →  {crtm.level_name(s)}")
227
228 print("thresholds for 'high':",
229       "linear τ4 =", round(sens.threshold(UN, 3, .9, "linear", 45, 120), 1), "min;",
230       "linear τ3 =", round(sens.threshold(UN, 2, .9, "linear", 25, 90), 1), "min;",
231       "anchored τ3 (τ4=45) =", round(sens.threshold(UN, 2, .9, "anchored", 1e-6, 24.99), 1), "min")
232 if HAVE_WIDGETS:
233     W.interact(times_demo, tau3=W.FloatSlider(value=25, min=0, max=200, step=0.5),
234                tau4=W.FloatSlider(value=45, min=0, max=260, step=0.5))
235 else:
236     times_demo()
237 F.fig_sensitivity(UN); plt.show()
238 """),
239     md("""
240 ## 6. Uncertainty in the weights (Section 7.5, Proposition 7)
241
242 w ~ Dirichlet(α·1). The exact mean is the equal-weight score and Var = s²/(qα+1).
243 Monte Carlo uses the article's seed 20261027 and 200 000 samples.
244 """),
245     code("""
246 rows = []
247 for meth in ("linear", "anchored"):
248     for a in (1, 4):
249         s = unc.summary(UN.vector(meth), a)
250         rows.append({"normalization": meth, "α": a, **{k: v for k, v in s.items() if k not in ("alpha", "levels
    ↪ ")}})
251 display(pd.DataFrame(rows))
252 F.fig_dirichlet(UN); plt.show()
253 """),
```

```
    md("## 7. Scenario profile and missing metrics (Section 7.6, Table 6)"),
    code("""
res = {m: crtm.evaluate_scenario(actions, method=m) for m in ("linear", "anchored")}
t6 = pd.DataFrame([{ "action": b, "goal": actions[b].goal, "q": actions[b].q, "κ": res["anchored"]["actions"][b].kappa,
                     "linear": r.score, "lvl": crtm.level_abbr(r.score), "lin. enclosure": (round(r.lower, 3), round(r.upper, 3)),
                     "anchored": res["anchored"]["actions"][b].score,
                     "lvl ": crtm.level_abbr(res["anchored"]["actions"][b].score),
                     "anc. enclosure": (round(res["anchored"]["actions"][b].lower, 3), round(res["anchored"]["actions"][b].upper, 3)),
                     "inconsistent (lin.)": len(r.inconsistent)} for b, r in res["linear"]["actions"].items()])
display(t6)
display(pd.DataFrame({m: {g: f"{v['mean']:.3f} / {v['min']:.3f}" for g, v in res[m]["profile"].items()} for m in res})
        .rename_axis("goal: mean / min"))
print("linear profile dominates anchored profile:", crtm.dominates(res["linear"]["profile"], res["anchored"]["profile"]))
F.fig_profile(actions); plt.show()
"""),
    md("""
## 8. Full reproduction check

Runs `scripts/reproduce.py`: compares the computed values with **107 values printed in the article**,
writes `results/results.json`, the tables (CSV) and all figures (PNG/SVG/PDF).
Set `FAST = True` for 20 000 instead of 200 000 Monte Carlo samples.
"""),
    code("""
FAST = False
import runpy, sys as _s
_s.argv = ["reproduce.py", "--out", str(ROOT / "results")] + (["--fast"] if FAST else [])
try:
    runpy.run_path(str(ROOT / "scripts" / "reproduce.py"), run_name="__main__")
except SystemExit as e:
    print("exit status:", e.code)
chk = pd.read_csv(ROOT / "results" / "checks.csv")
print(f"{chk.ok.sum()} / {len(chk)} published values reproduced")
display(chk[~chk.ok] if (~chk.ok).any() else Markdown("**All published values reproduced.**"))
"""),
    code("""
# optional: download the results as a zip (Colab only)
import shutil
zip_path = shutil.make_archive(str(ROOT / "crtm_results"), "zip", ROOT / "results")
try:
    from google.colab import files
    files.download(zip_path)
except ImportError:
    print("results archived at", zip_path)
"""),
    md("""
## 9. Sandbox: score your own metric

Enter a class table in the printed order critical / below average / average / above average / high
(ξ₀ … ξ₅, see Section 5.1 for the encoding rule) and a measured value.
"""),
    code("""
def my_metric(xi0=180., xi1=179., xi2=90., xi3=30., xi4=10., xi5=0., value=25.):
    try:
        t = crtm.ClassTable((xi0, xi1, xi2, xi3, xi4, xi5))
    except crtm.ClassTableError as e:
        print("invalid class table:", e); return
    lt, ln, an = t.tabulated_level(value), t.linear(value), t.anchored(value)
    print(f"polarity σ = {t.sigma:+d}; tabulated level: {crtm.LEVEL_NAMES[lt-1]}")
    print(f"linear   n_ch = {ln:.4f} → {crtm.level_name(ln)}{'   ✗ inconsistent' if crtm.level(ln) != lt else '}")
    print(f"anchored ν_ξ  = {an:.4f} → {crtm.level_name(an)}")
    print("Corollary 1 interval (class 4 → 'high' under n_ch):", t.corollary1_interval()["x_interval"])

if HAVE_WIDGETS:
    W.interact_manual(my_metric, **{k: W.FloatText(v) for k, v in
        dict(xi0=180., xi1=179., xi2=90., xi3=30., xi4=10., xi5=0., value=25.).items()})
else:
    my_metric()
"""),
    md("""
---
**License.** Code: MIT. Data and figures: CC BY 4.0. Please cite the article and the Zenodo record
(see `CITATION.cff`). Generated from the sources by `tools/build_notebook.py`.
"""),
]


def main():
    nb = nbf.v4.new_notebook()
    nb.cells = CELLS
    nb.metadata = {"kernelspec": {"display_name": "Python 3", "language": "python", "name": "python3"},
```

```
                    "language_info": {"name": "python"},
                    "colab": {"provenance": [], "toc_visible": True, "name": "CRTM_Colab.ipynb"}}
    out = ROOT / "notebooks" / "CRTM_Colab.ipynb"
    nbf.write(nb, out)
    print("wrote", out, f"({out.stat().st_size / 1024:.0f} KiB)")


if __name__ == "__main__":
    main()
```